
IMAGE PROCESSING AND COMPUTER VISION

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

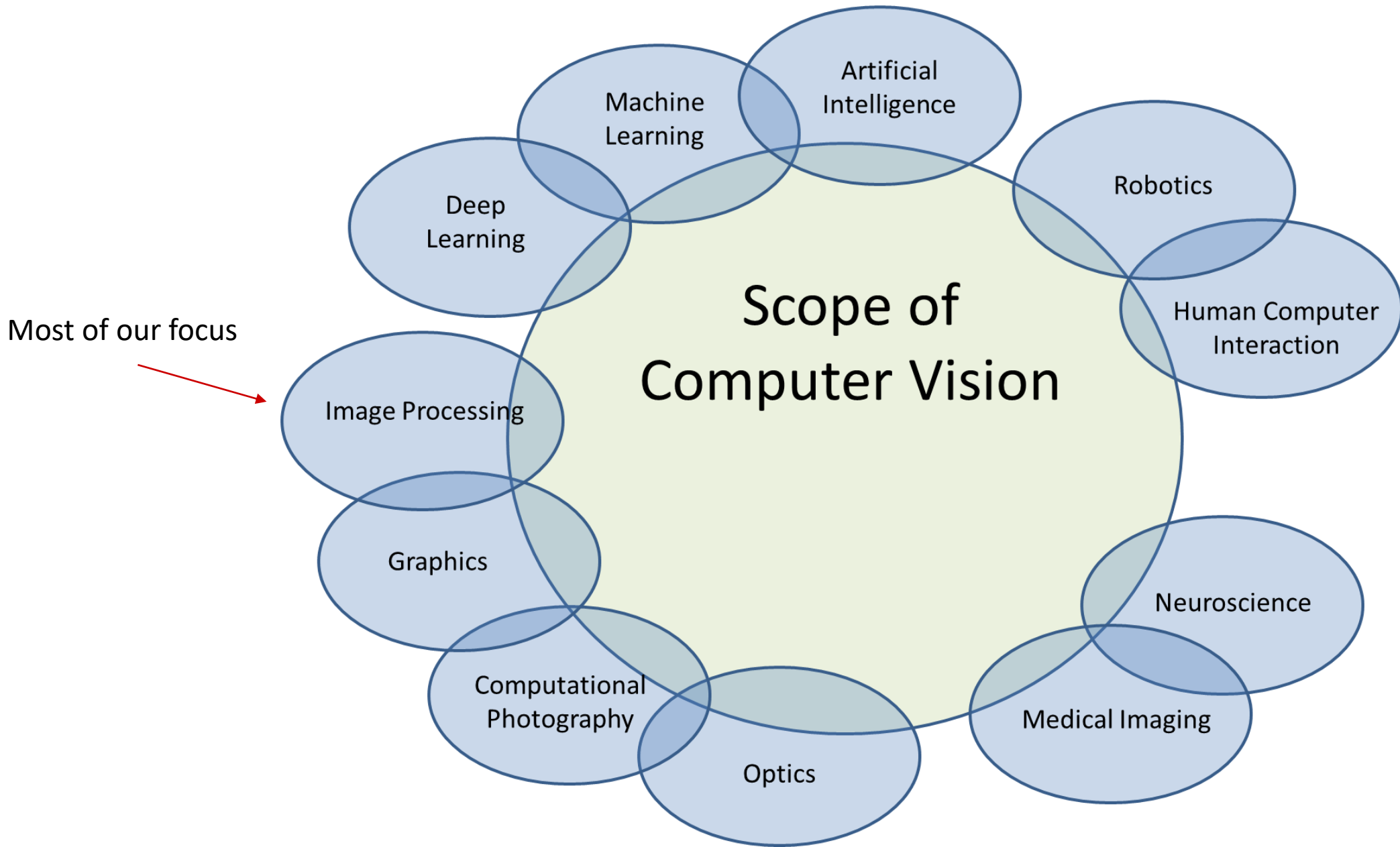
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CSCI 1430 see [link](#)
- Some content adapted from Dr. ElSayed El-Alfy

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- IPVC Intro
- Image Processing Tasks:
 - Image Filtering
 - Edge detection
 - Template Matching
 - Advanced Templated Matching – Object Recognition/Classification
- Summary

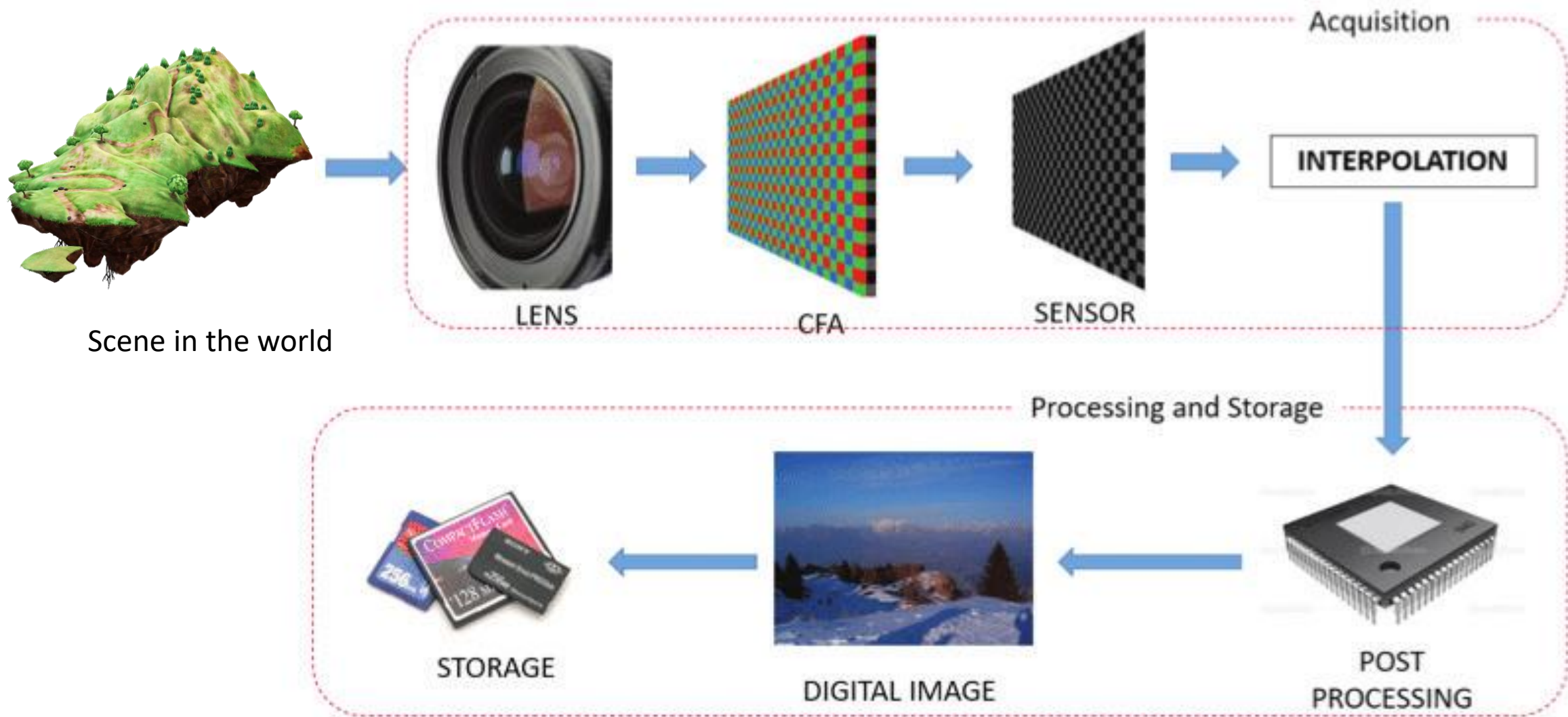


IPCV Intro

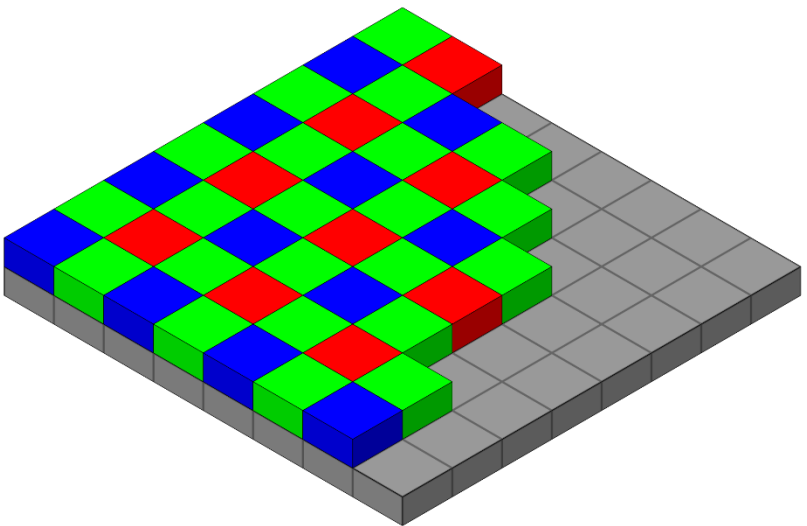
Computer Vision and Image Processing

- “Computer vision is an interdisciplinary scientific field that deals with how computers can gain high-level understanding from **digital images or videos**. From the perspective of engineering, it seeks to understand and automate tasks that the human visual system can do.” – from [Wikipedia](#)
- “Computer vision tasks include methods for acquiring, processing, analyzing and understanding **digital images**” – from [Wikipedia](#)
- Our main focus is going to be image processing.
- In particular, we will discuss a fundamental technique: **filters/kernels**
- See how filters can be used to do a wide range of tasks:
 - Edge detection
 - Feature/template detection
 - Image/Object classification (supervised machine learning)

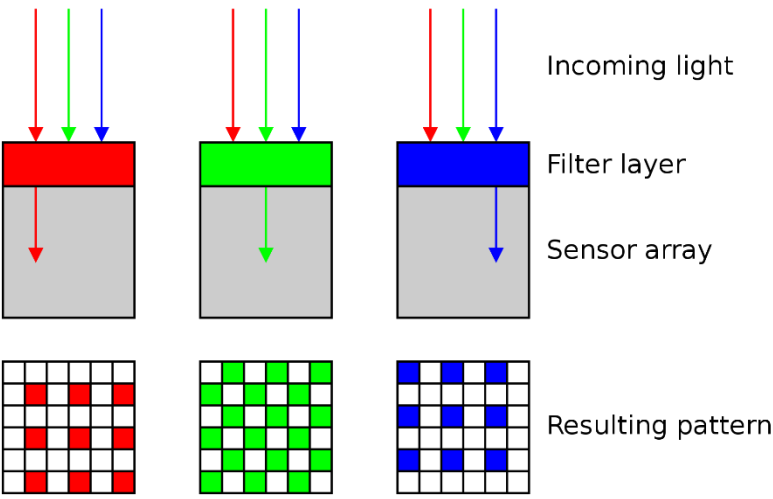
Capturing an Image



Galvan, Fausto & Battiato, Sebastiano. (2019). Image/Video Forensics: Theoretical Backgrounds, Methods and Best Practices – Part two: from analog to digital world. 45-50.



Color filter array over pixel sensor

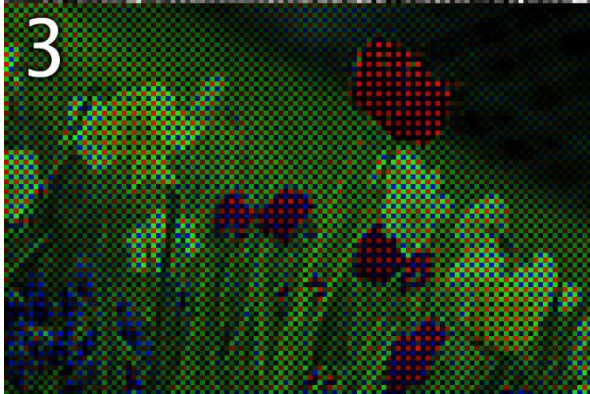


Bayer color filter

Intensity values
captured by sensor



Intensity-to-color
captured by sensor

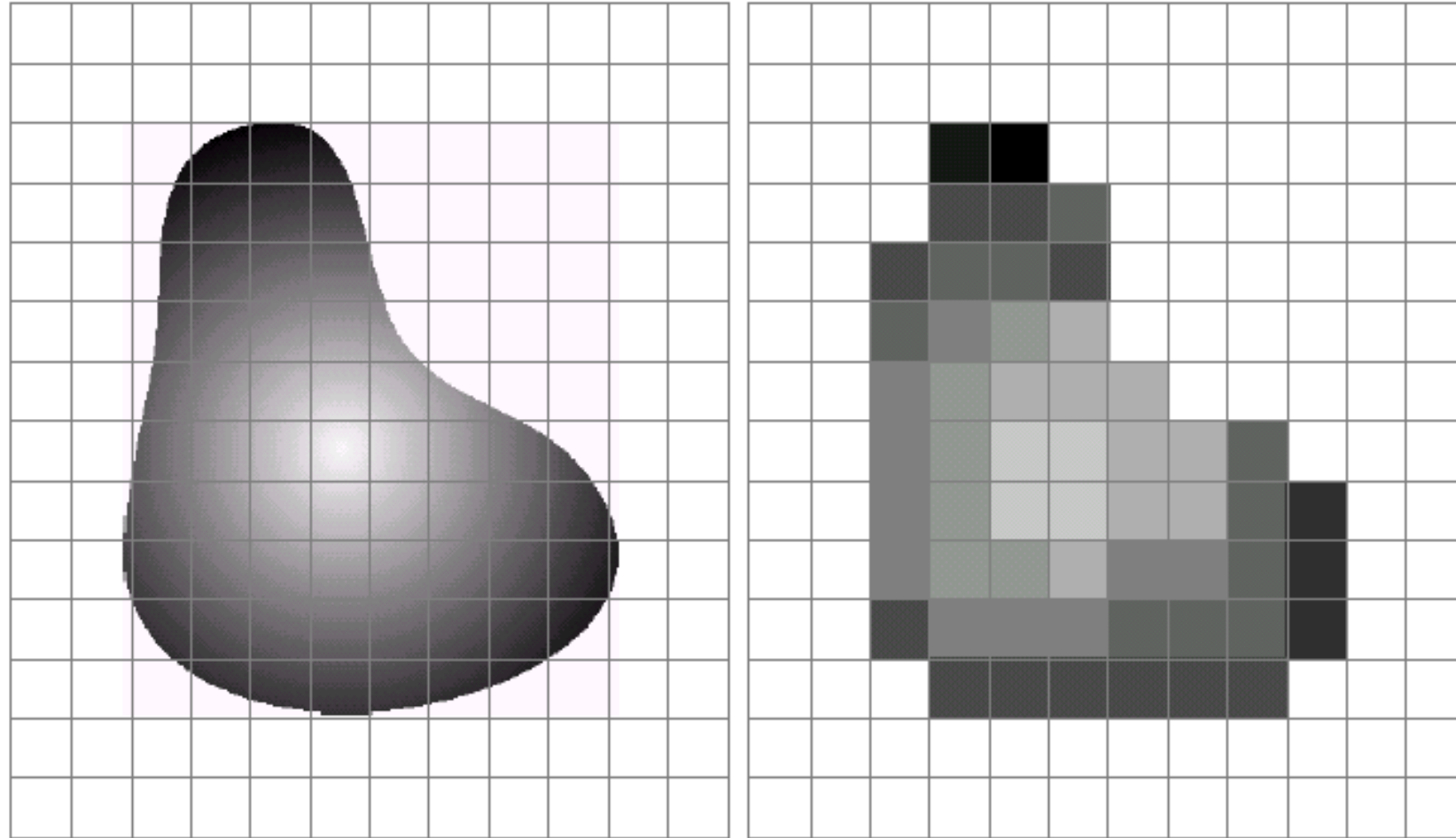


Reconstructed image
after computational
interpolation



Images from: [link1](#)

Quantization/Digitization is lossy for shape and intensity

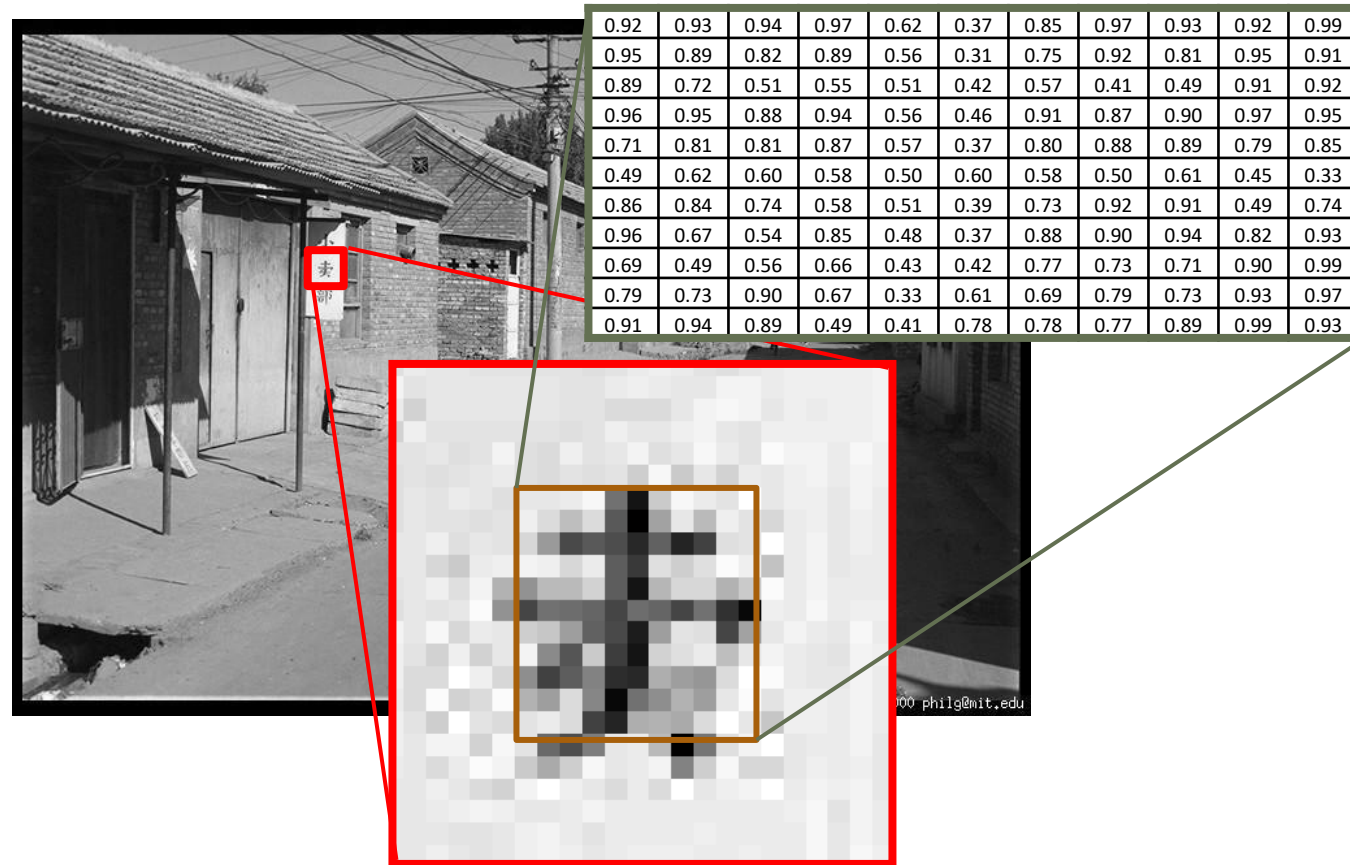


a b

FIGURE 2.17 (a) Continuous image projected onto a sensor array. (b) Result of image sampling and quantization.

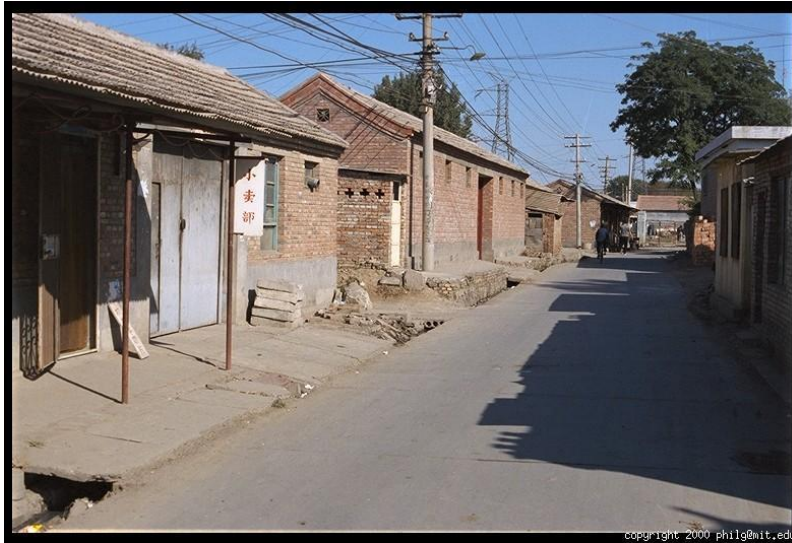
Grayscale intensity

- Black and white image grayscale image.
- Smaller numbers near 0 represent the darker shades
- Larger numbers near 1 represent the lighter or the white shade (higher intensity of light).



Color images

- Color images have channels for different colors of light.
- Most common is the three channels RGB: red, green, blue.
- When visualizing, the three channels are used to reconstruct the ‘full’ color single image.



Red intensity



Green



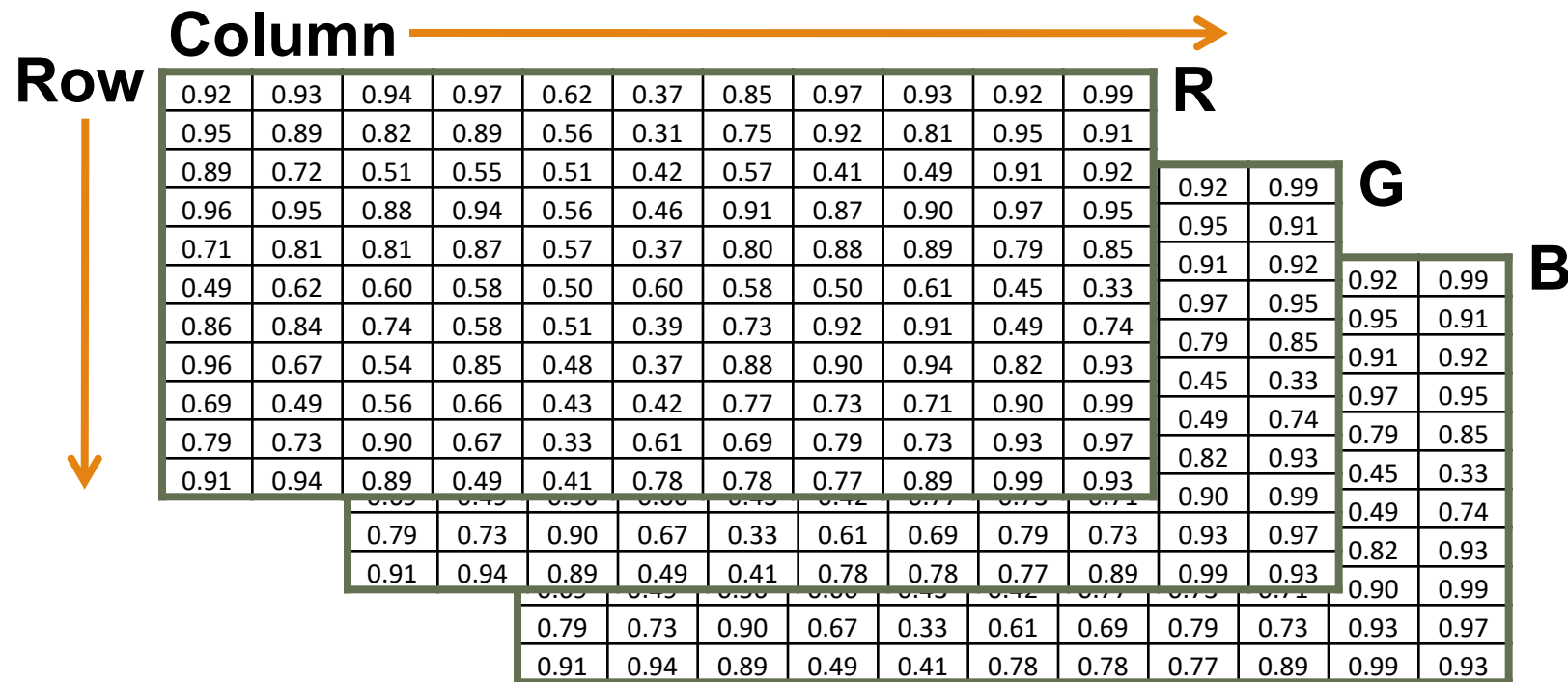
Blue



Images in Python Numpy

$N \times M$ **RGB** image “im”

- `im[0,0,0]` = top-left pixel value in R-channel
- `im[y, x, c]` = y pixels down, x pixels to right, in the c^{th} channel
- `im[N-1, M-1, 2]` = bottom-right pixel in B-channel

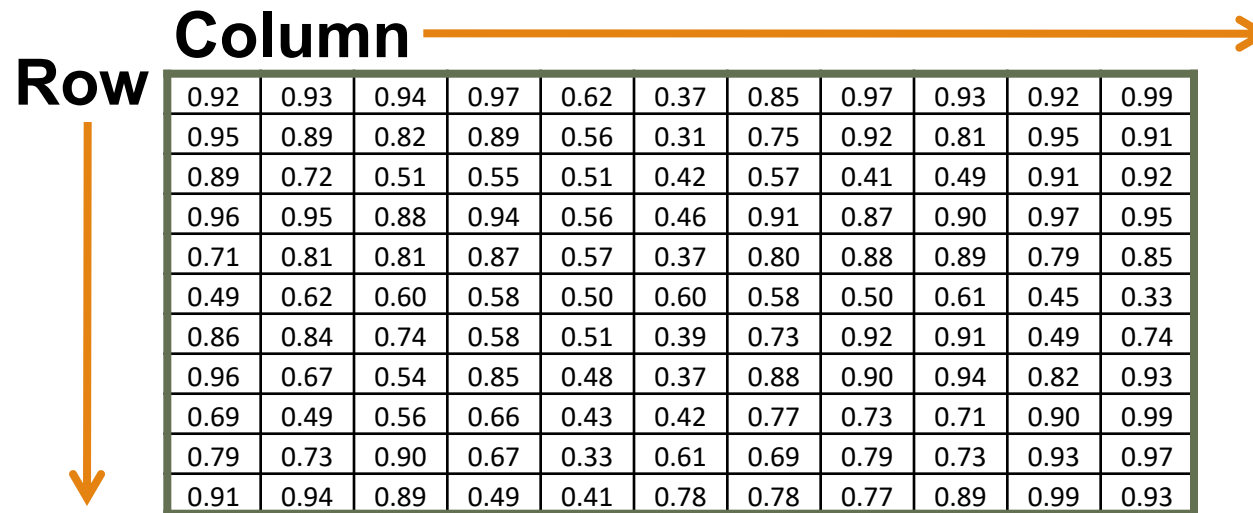


Images in Python Numpy

Simple to use image processing python package: <https://scikit-image.org/>

Take care between types!

- uint8 (values 0 to 255) – `io.imread("file.jpg")`
- float32 (values 0 to 255) – `io.imread("file.jpg").astype(np.float32)`
- float32 (values 0 to 1) – `img_as_float32(io.imread("file.jpg"))`



Row	Column	0.92	0.93	0.94	0.97	0.62	0.37	0.85	0.97	0.93	0.92	0.99
		0.95	0.89	0.82	0.89	0.56	0.31	0.75	0.92	0.81	0.95	0.91
		0.89	0.72	0.51	0.55	0.51	0.42	0.57	0.41	0.49	0.91	0.92
		0.96	0.95	0.88	0.94	0.56	0.46	0.91	0.87	0.90	0.97	0.95
		0.71	0.81	0.81	0.87	0.57	0.37	0.80	0.88	0.89	0.79	0.85
		0.49	0.62	0.60	0.58	0.50	0.60	0.58	0.50	0.61	0.45	0.33
		0.86	0.84	0.74	0.58	0.51	0.39	0.73	0.92	0.91	0.49	0.74
		0.96	0.67	0.54	0.85	0.48	0.37	0.88	0.90	0.94	0.82	0.93
		0.69	0.49	0.56	0.66	0.43	0.42	0.77	0.73	0.71	0.90	0.99
		0.79	0.73	0.90	0.67	0.33	0.61	0.69	0.79	0.73	0.93	0.97
		0.91	0.94	0.89	0.49	0.41	0.78	0.78	0.77	0.89	0.99	0.93

Image Filters (kernels)

Image filtering – Cross-correlation Operation

- “Applying a filter” is a fundamental technique in image processing. It can be used as a basic tool for a wide variety of image processing algorithms.

Apply a **filter** f (also called kernel) on an **input image** I to produce a **new image** h .

The filter is computing a local neighborhood function at each position:

$$\begin{array}{l} \text{h=output} \qquad \text{f=filter} \qquad \text{I=image} \\ h[m,n] = \sum_{k,l} f[k,l] \ I[m+k,n+l] \end{array}$$

m, n range over the pixels in the input image I

k, l range over the pixels in the filter matrix f

Example: Box Filter

$$f[\cdot, \cdot]$$

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Note we could have written this filter as 1/9 in each element. This is just notation for *some* filters.

$$h[m, n] = \sum_{k, l} f[k, l] I[m + k, n + l]$$

Slide credit: David Lowe (UBC)

Example of applying a filter [1]

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$I[.,.]$

$h[.,.]$

Position the filter centered at m, n then multiply each pixel of f with corresponding pixel of I , then sum.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$$m = 1, n = 1$$
$$k, l = [-1,0,1]$$

Credit: S. Seitz

Example of applying a filter [2]

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10							

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$$\begin{aligned} m &= 2, n = 1 \\ k,l &= [-1,0,1] \end{aligned}$$

Credit: S. Seitz

Example of applying a filter [3]

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20						

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$$\begin{aligned} m &= 3, n = 1 \\ k,l &= [-1,0,1] \end{aligned}$$

Credit: S. Seitz

Example of applying a filter [4]

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30					

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$m = 4, n = 1$
 $k, l = [-1,0,1]$

Credit: S. Seitz

Example of applying a filter [5]

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$m = 5, n = 1$
 $k, l = [-1,0,1]$

Credit: S. Seitz

Example of applying a filter [6]

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$$I[\cdot, \cdot]$$
[illegible]
$$h[\cdot, \cdot]$$
[illegible]

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$$\begin{array}{l} m = 4, n = 6 \\ k, l = [-1, 0, 1] \end{array}$$

Credit: S. Seitz

Example of applying a filter [7]

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				
						?			
				50					

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$m = 6, n = 4$
 $k, l = [-1,0,1]$

Credit: S. Seitz

Example of applying a filter [8]

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$$I[.,.]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[.,.]$$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

Credit: S. Seitz

Box Filter

What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieve **smoothing/blur** effect (remove 'sharp' features)

$$f[\cdot, \cdot]$$

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Slide credit: David Lowe (UBC)

Smoothing/blur with box filter

$$f[\cdot, \cdot]$$

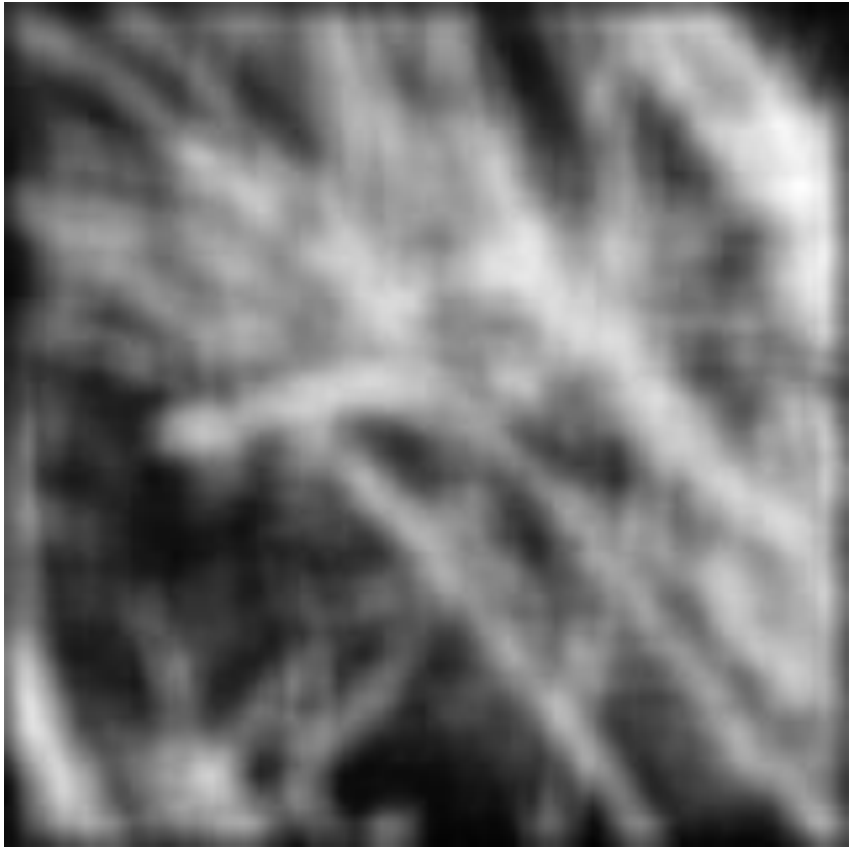
0.25	0.25	0.25
0.25	0.25	0.25
0.25	0.25	0.25

$$I[\cdot, \cdot]$$



=

$$h[\cdot, \cdot]$$



David Lowe

Using Image filtering

- By defining various filters, we can:
 - Enhance images: Denoise, resize, increase contrast, etc.
 - Extract information from images: Texture, edges, distinctive points, etc.
 - Detect patterns: Template matching
- We will cover edge detection and template matching.

Apply the filter to the image. What size should the output image be?
How is filter actually applied?

0.1	0.1	0.1
0.1	0.1	0.1
0.1	0.1	0.1

Filter/Kernel
3x3

5	1	9	3	3
7	9	2	5	7
1	7	2	8	6
3	6	4	8	1
4	9	3	3	6

Image
5x5



Result Image

5x5?

7x7?

3x3?

There are three common modes: valid, same, and full.

Technical detail: filtering mode [2]

Applying filter is known as *cross-correlation*. $h = f \otimes I$

Convolution is also related to cross-correlation; won't go into details, but it is *slightly* different. Convolution is the term used in deep learning. $h = f * I$

For purposes of this course, we will just discuss applying filters and how its done. We'll regard cross-correlation and convolution as the same operation.

See
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.correlate2d.html>

And
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.convolve2d.html>

`scipy.signal.correlate2d(in1, in2, mode='full', boundary='fill', fillvalue=0)` [\[source\]](#)

Cross-correlate two 2-dimensional arrays.

Cross correlate *in1* and *in2* with output size determined by *mode*, and boundary conditions determined by *boundary* and *fillvalue*.

Parameters: **in1** : *array_like*

First input.

in2 : *array_like*

Second input. Should have the same number of dimensions as *in1*.

mode : *str* {'full', 'valid', 'same'}, optional

A string indicating the size of the output:

full

The output is the full discrete linear cross-correlation of the inputs. (Default)

valid

The output consists only of those elements that do not rely on the zero-padding. In 'valid' mode, either *in1* or *in2* must be at least as large as the other in every dimension.

same

The output is the same size as *in1*, centered with respect to the 'full' output.

boundary : *str* {'fill', 'wrap', 'symm'}, optional

A flag indicating how to handle boundaries:

fill

pad input arrays with fillvalue. (default)

wrap

circular boundary conditions.

symm

symmetrical boundary conditions.

fillvalue : *scalar*, optional

In valid mode, you apply the filter **within** the dimensions of the image. The resulting image will have a reduced size.

051	011	091	3	3
071	091	021	5	7
011	071	021	8	6
3	6	4	8	1
4	9	3	3	6

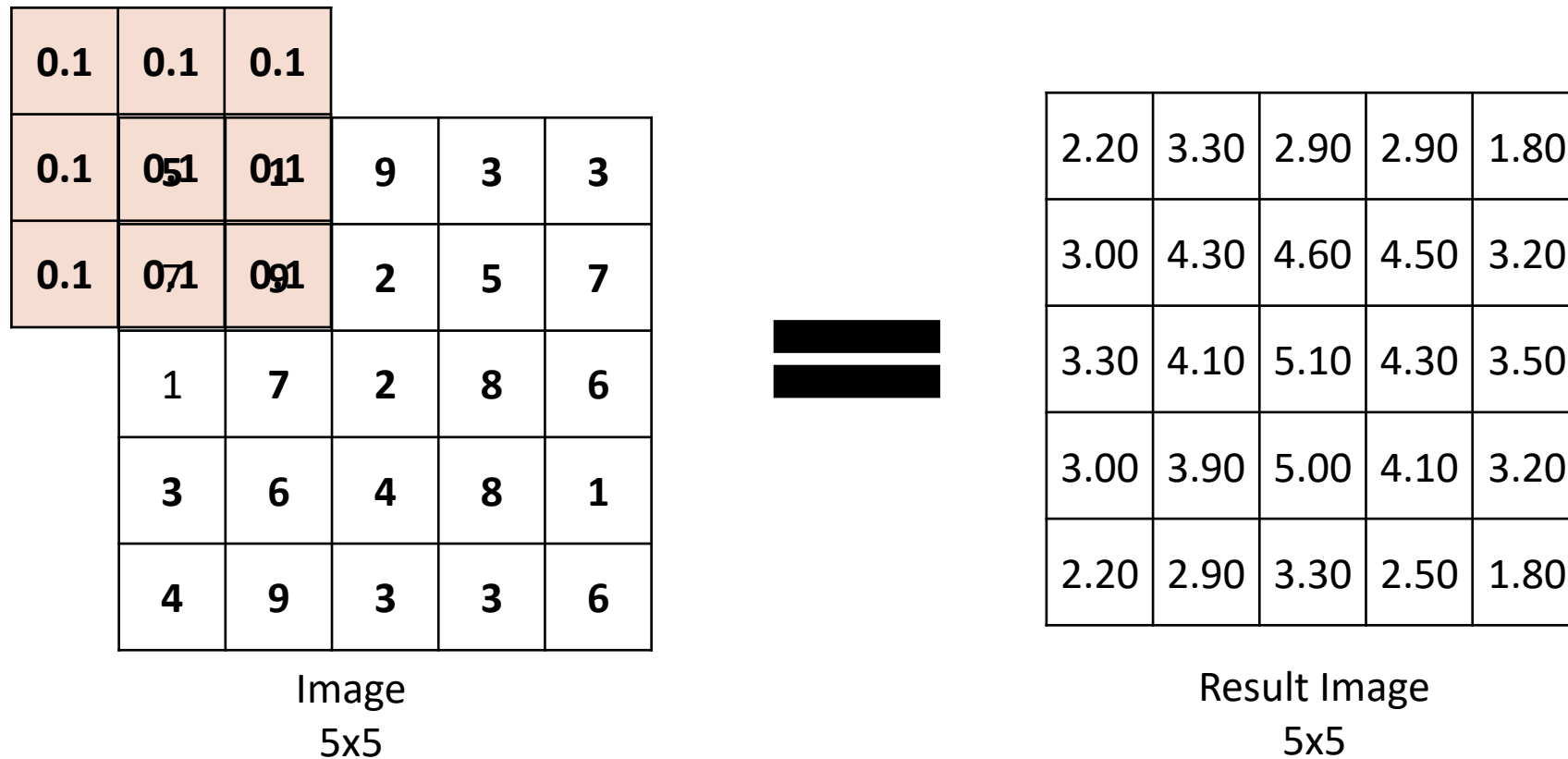
Image
5x5

=

4.30	4.60	4.50
4.10	5.10	4.30
3.90	5.00	4.10

Result Image
3x3

In same mode, the size of the output is the same as the image. Pad the outer parts of the image with 0; filter is positioned such that it is **centralized** at the edges.



In full mode, apply the filter to the entire image including the edge positions. Pad outer pixels with 0; filter is positioned “edge-most” with the image.

0.1	0.1	0.1				
0.1	0.1	0.1				
0.1	0.1	0.1	1	9	3	3
			7	9	2	5
			1	7	2	8
			3	6	4	8
			4	9	3	3

Image
5x5



0.50	0.60	1.50	1.30	1.50	0.60	0.30
1.20	2.20	3.30	2.90	2.90	1.80	1.00
1.30	3.00	4.30	4.60	4.50	3.20	1.60
1.10	3.30	4.10	5.10	4.30	3.50	1.40
0.80	3.00	3.90	5.00	4.10	3.20	1.30
0.70	2.20	2.90	3.30	2.50	1.80	0.70
0.40	1.30	1.60	1.50	1.20	0.90	0.60

Result Image
7x7

```

import numpy as np
import scipy.signal as ssig

filter = (1/10)*np.array([[1, 1, 1],
                           [1, 1, 1],
                           [1, 1, 1]])
image = np.array([[5,1,9,3,3],
                  [7,9,2,5,7],
                  [1,7,2,8,6],
                  [3,6,4,8,1],
                  [4,9,3,3,6]])

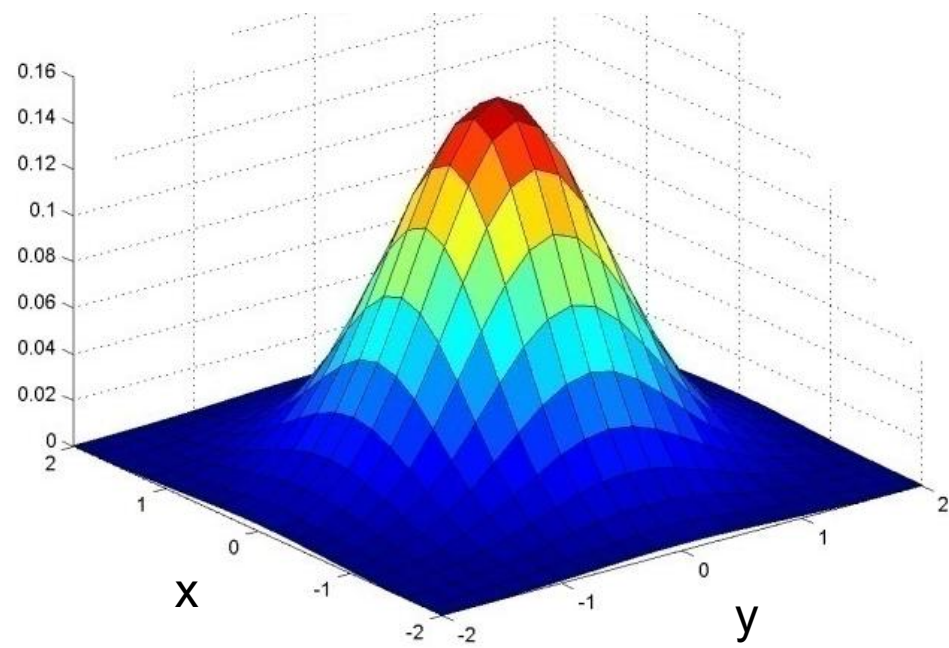
full_res = ssig.correlate2d(image, filter, mode='full')
valid_res = ssig.correlate2d(image, filter, mode='valid')
same_res = ssig.correlate2d(image, filter, mode='same')

print(full_res, full_res.shape)
print('-----')
print(valid_res, valid_res.shape)
print('-----')
print(same_res, same_res.shape)
print('-----')

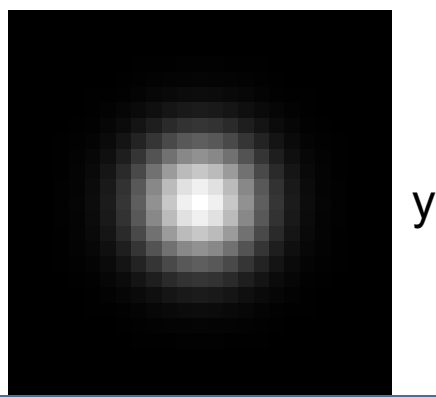
```

Important filters: Gaussian Filter (used for smooth/blur effect)

Weight contributions of neighboring pixels by nearness



Viewed
from top



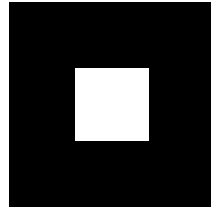
x					y
0.003	0.013	0.022	0.013	0.003	
0.013	0.059	0.097	0.059	0.013	
0.022	0.097	0.159	0.097	0.022	
0.013	0.059	0.097	0.059	0.013	
0.003	0.013	0.022	0.013	0.003	

Kernel size 5 x 5,
Standard deviation $\sigma = 1$

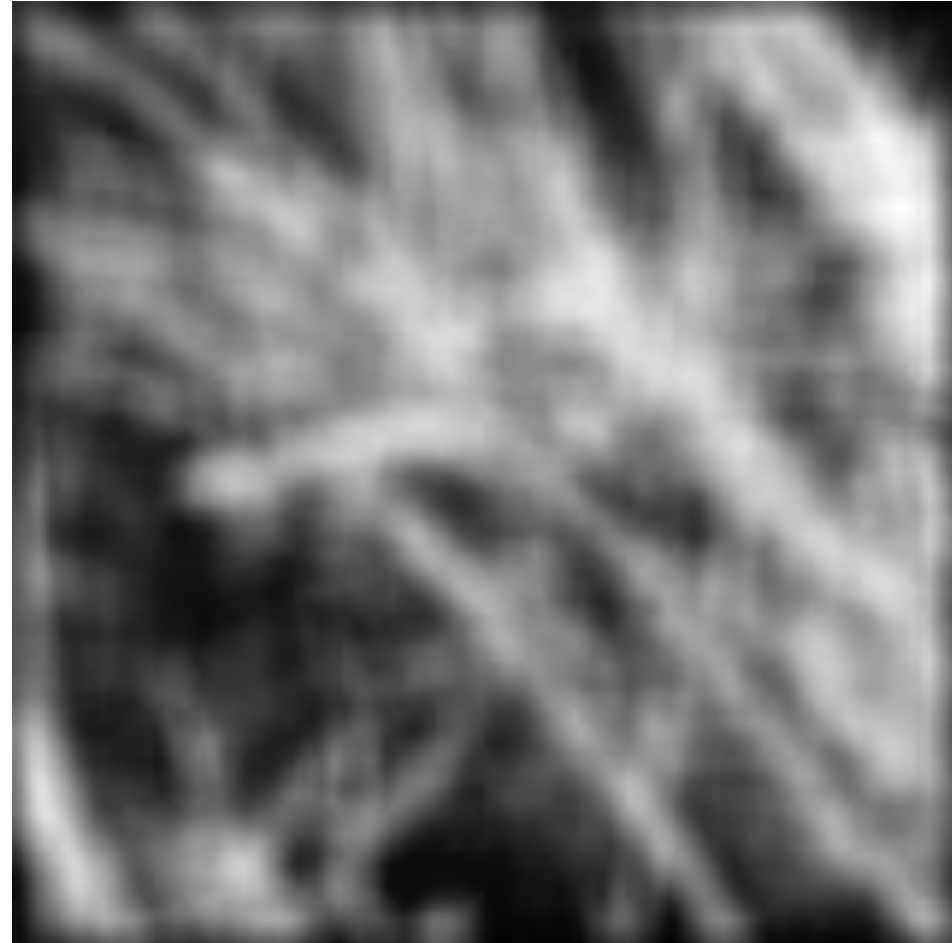
$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Slide credit: Christopher Rasmussen

Smoothing with box filter

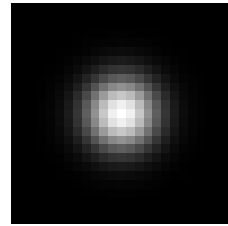


0.1	0.1	0.1
0.1	0.1	0.1
0.1	0.1	0.1

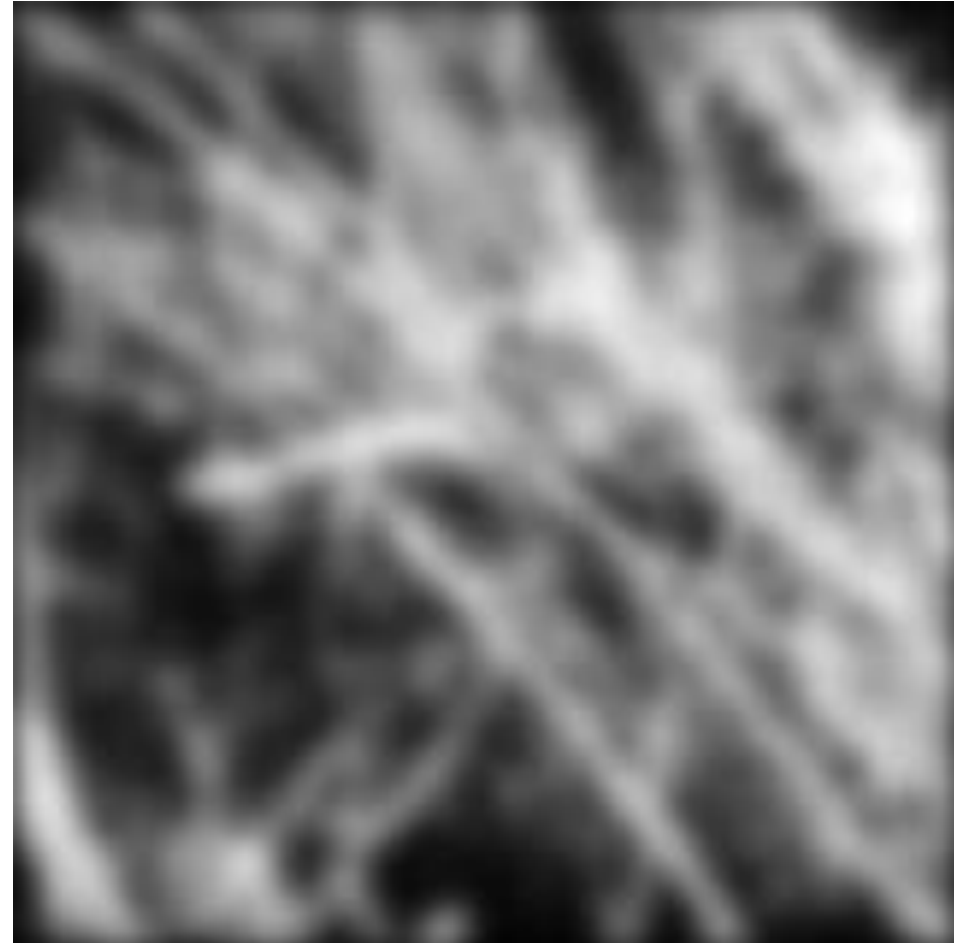


James Hays

Smoothing with Gaussian filter – better results

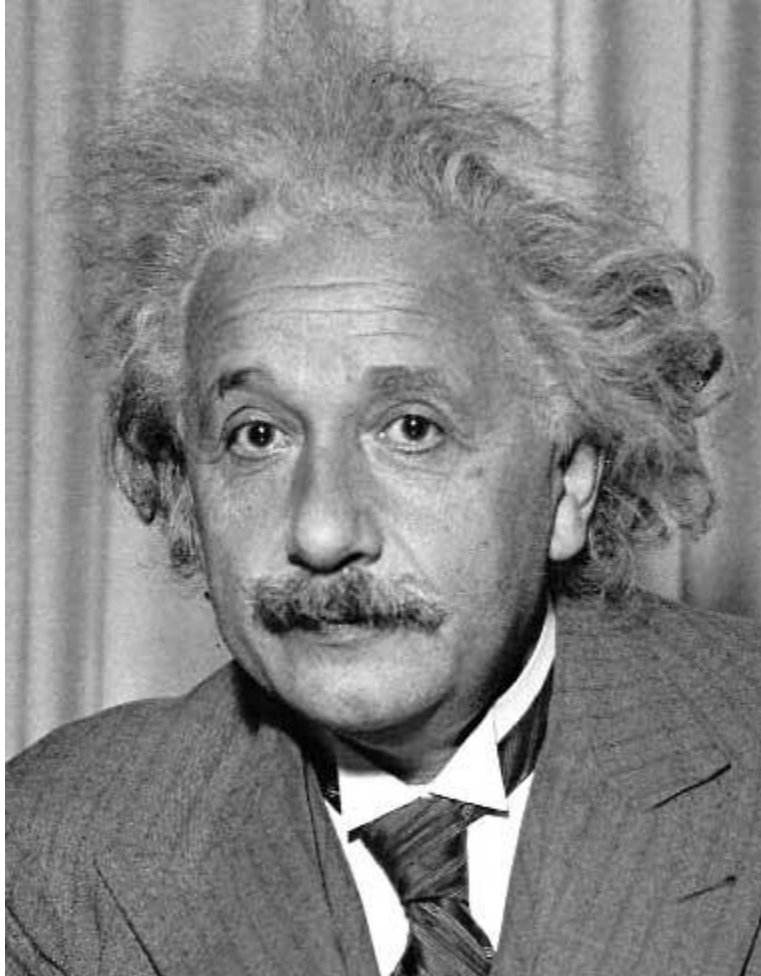


0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003



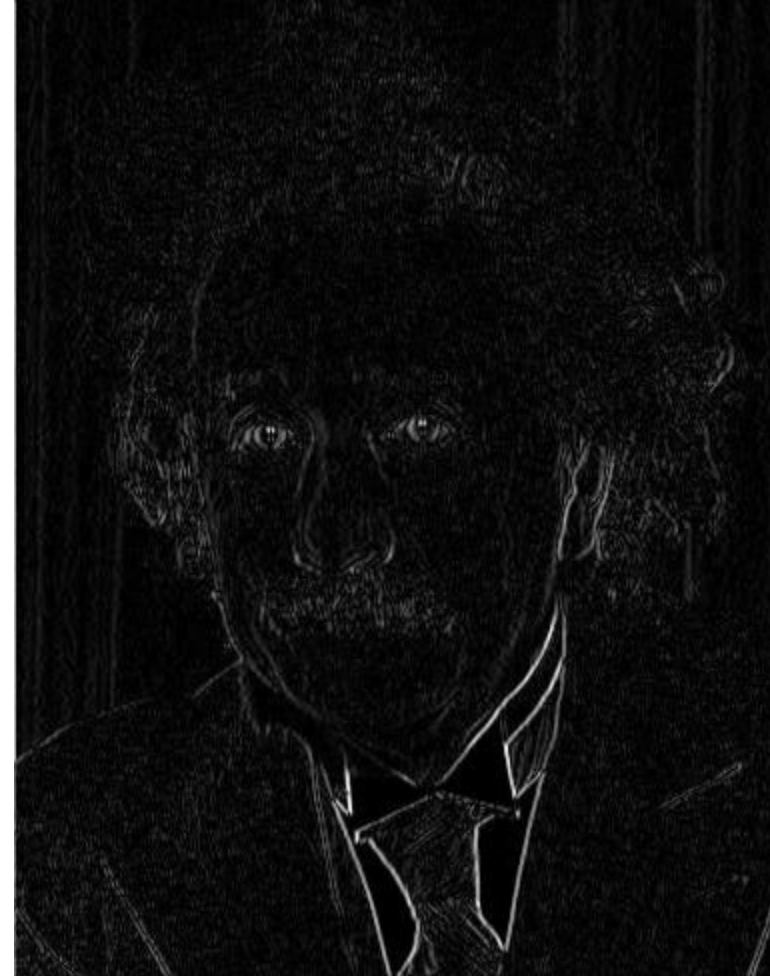
James Hays

Important filters: Edge filters



1	0	-1
2	0	-2
1	0	-1

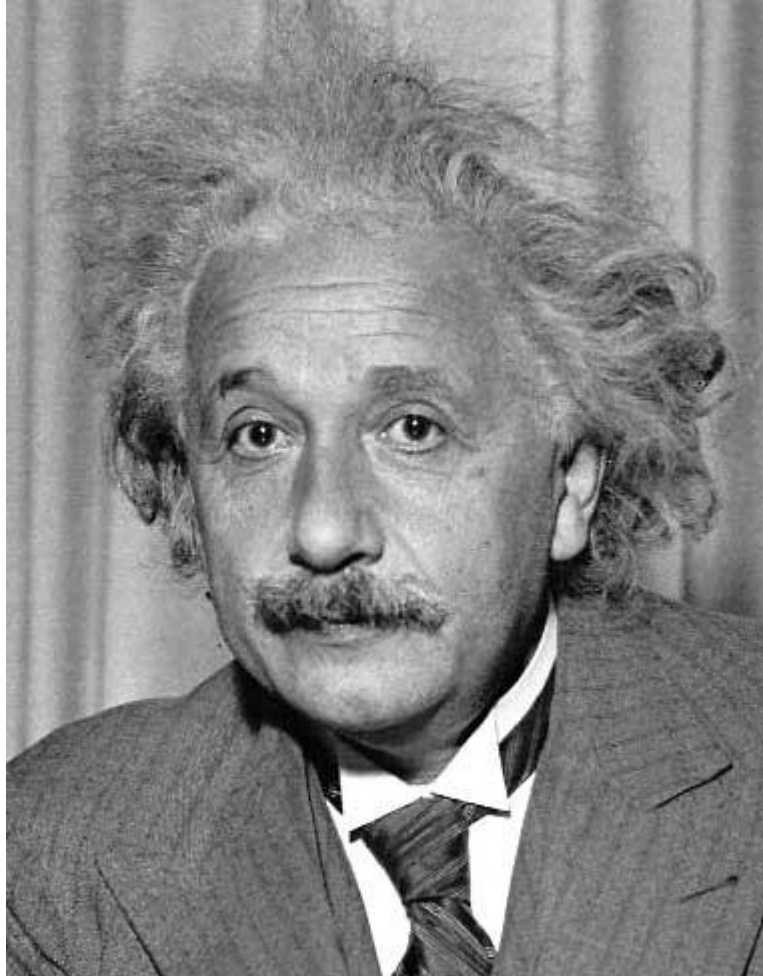
Sobel



Vertical Edge
(absolute value)

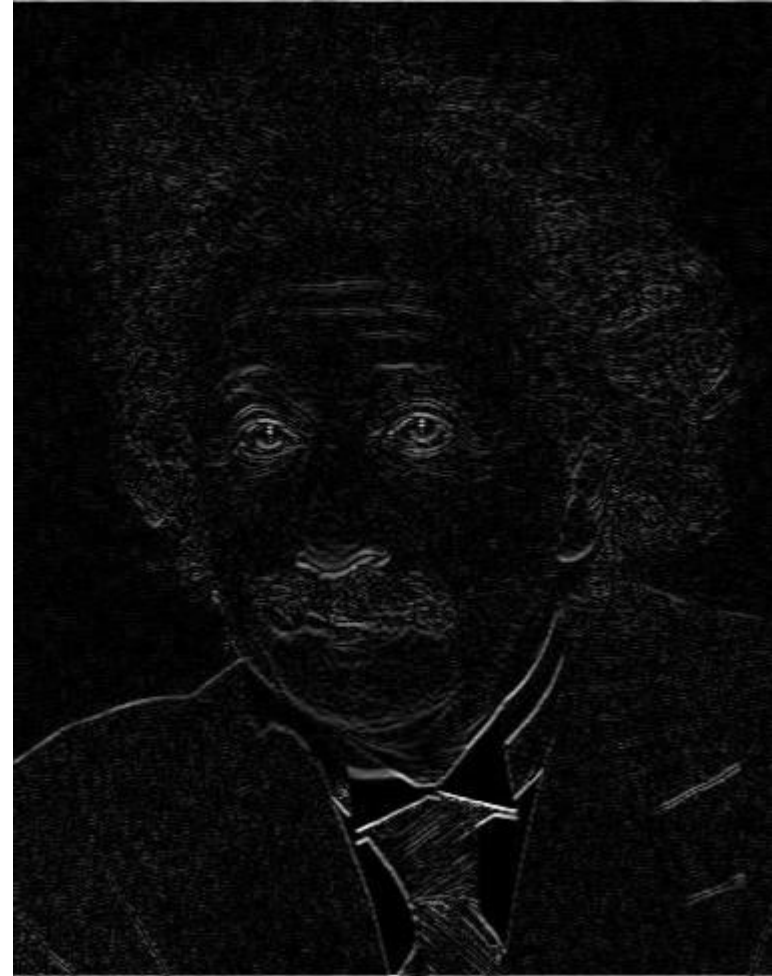
David Lowe

Important filters: Edge filters



1	2	1
0	0	0
-1	-2	-1

Sobel



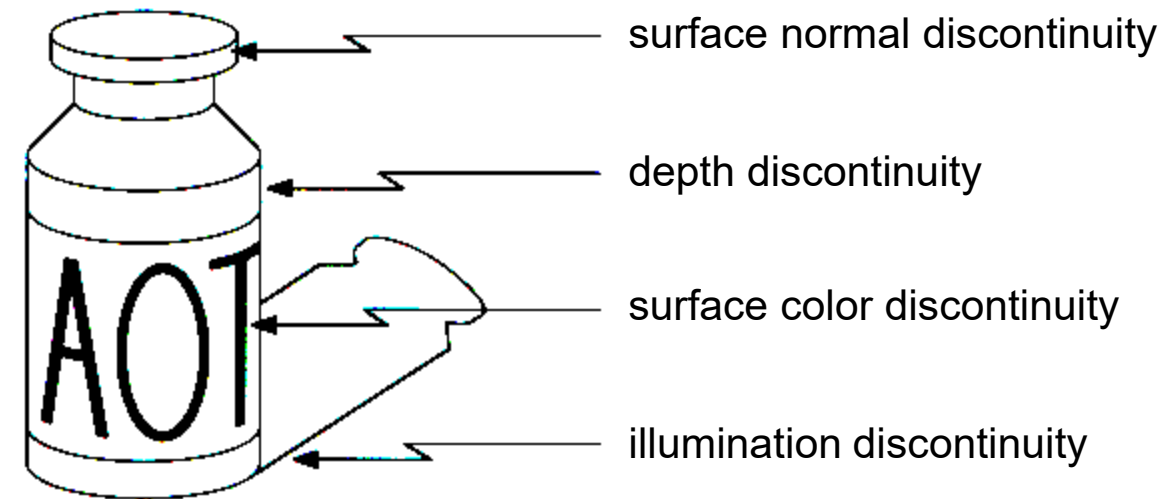
Horizontal Edge
(absolute value)

David Lowe

Edge Detection

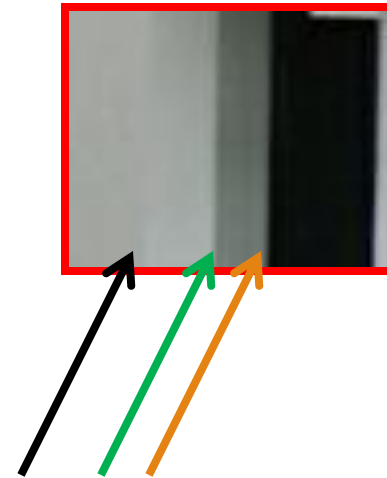
Edge Detection

- Goal: Identify visual changes (discontinuities) in an image.
- Intuitively, semantic information is encoded in edges.
- Edges are caused by a variety of factors
- Why do we care about edges?
 - Extract info and recognize objects.
 - Help recover geometry and viewpoints.
 - Can be used as part of object detection program.



Source: Steve Seitz

Closeup of edges



Source: D. Hoiem

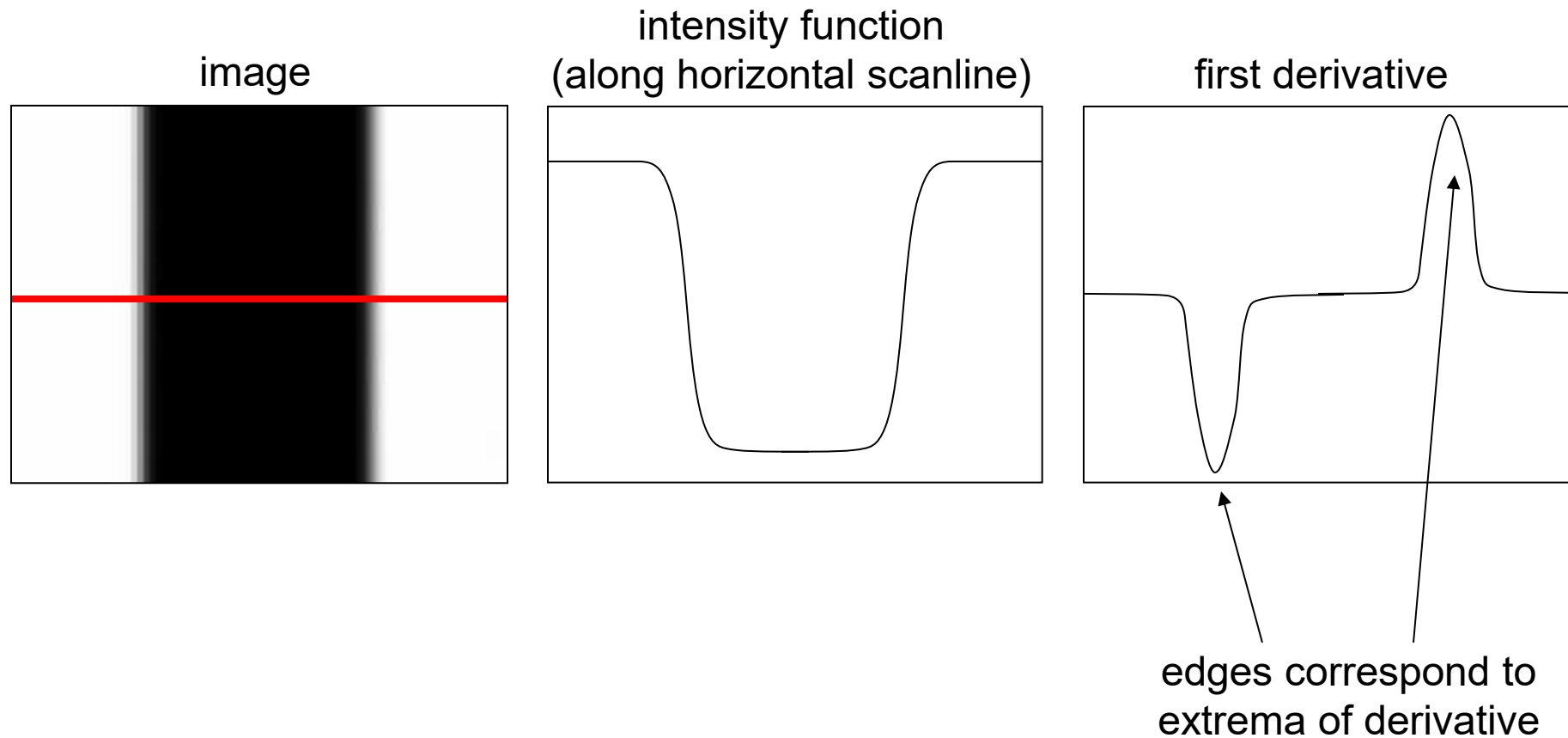
Closeup of edges



Source: D. Hoiem

Characterizing Edges

- An edge is a place of **rapid change** in the image intensity function



Calculating discrete first derivative

- From calculus, the continuous derivative at the point x is the limit of the central difference:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + 0.5h) - f(x - 0.5h)}{h}$$

How do we compute the derivative of a discrete signal?

10	20	10	200	210	250	250
----	----	----	-----	-----	-----	-----



$$f'(x) = \frac{f(x+1) - f(x-1)}{2} = \frac{210 - 10}{2} = 100$$

-1	0	1
----	---	---

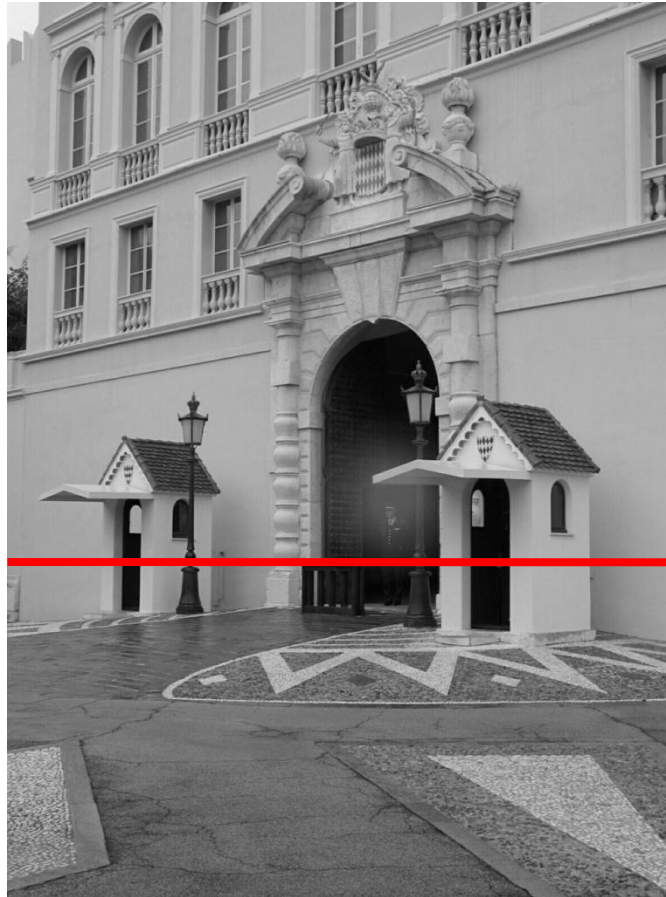
1D derivative filter

Adapted from: [link](#)

Intensity Profile without noise

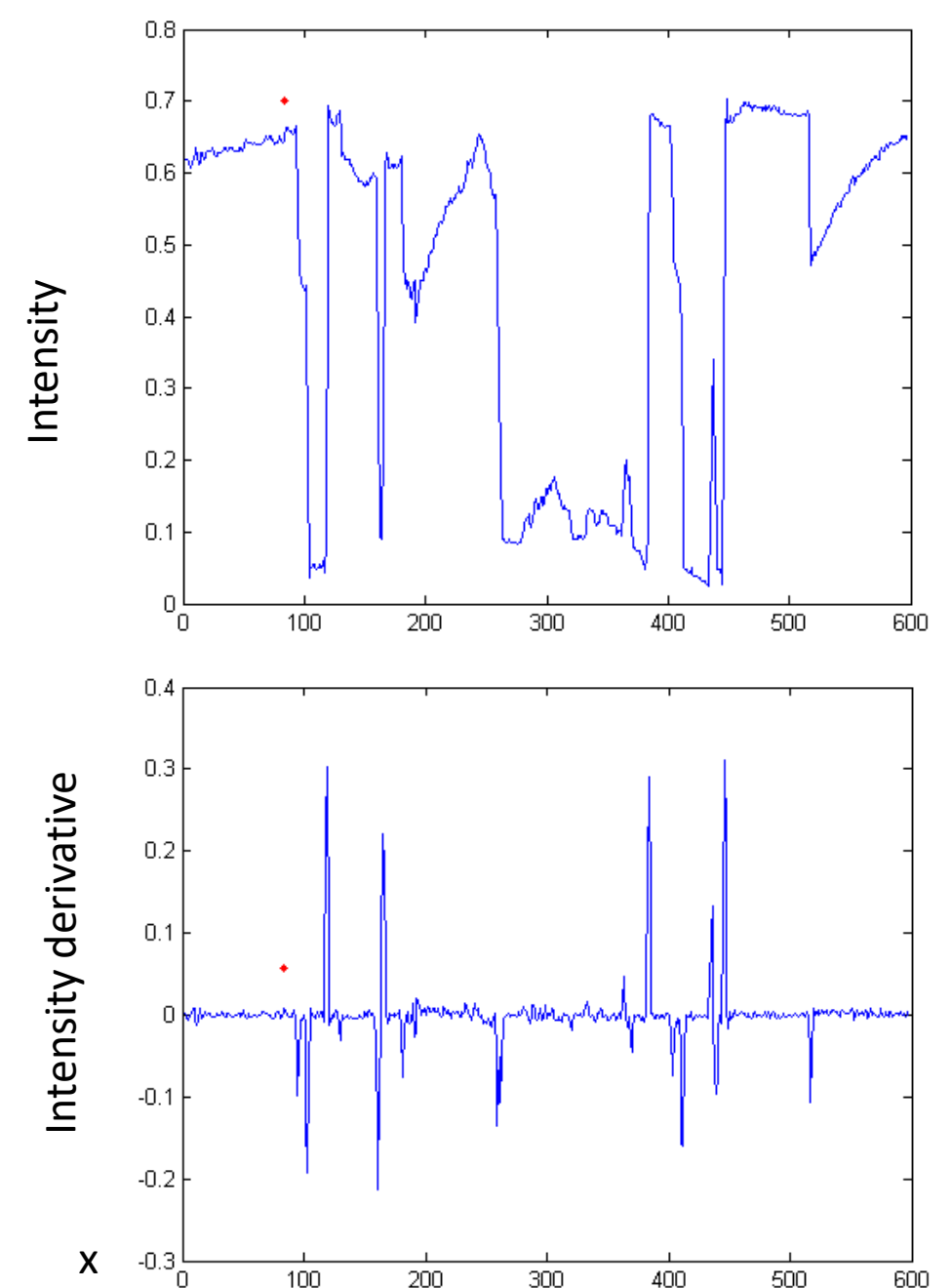
Consider if we look at the horizontal line of pixels in red, and we plot the intensity (top plot) and derivative (bottom plot).

The significant change in signal intensity may indicate an edge.

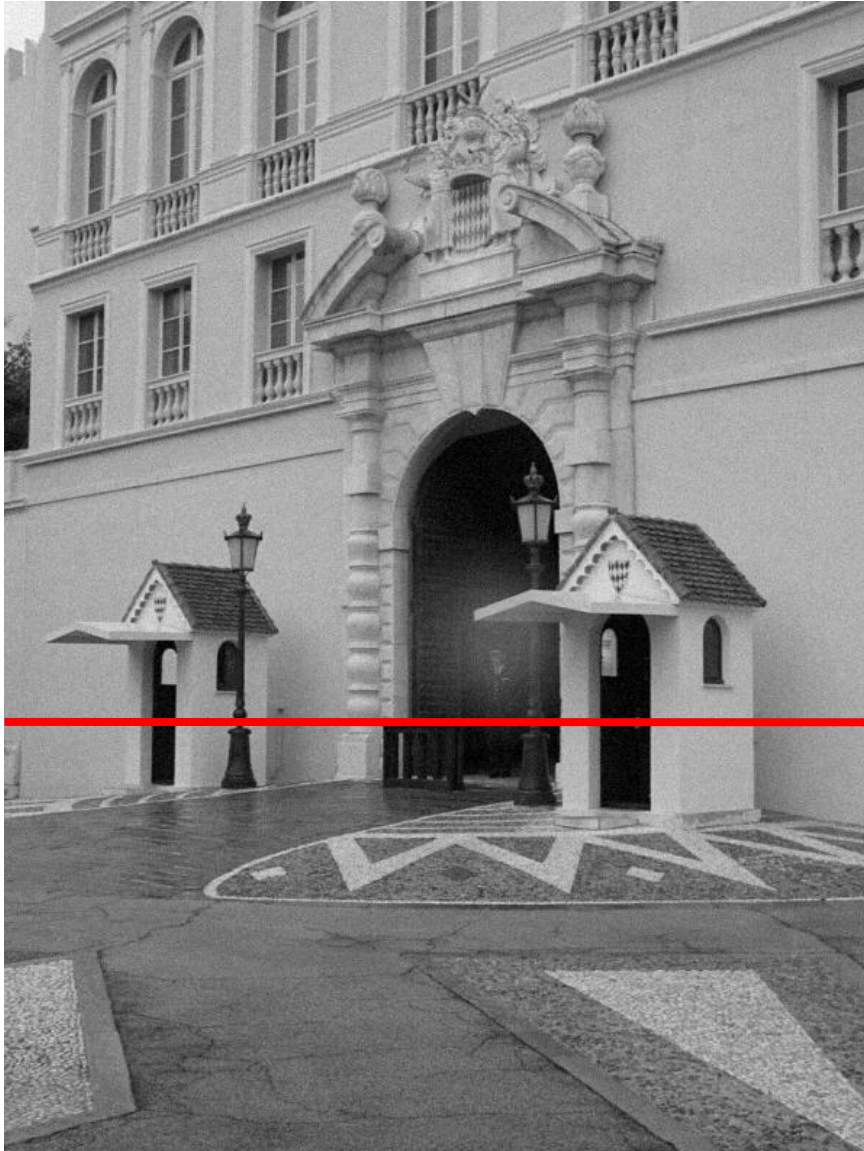


600 pixels wide

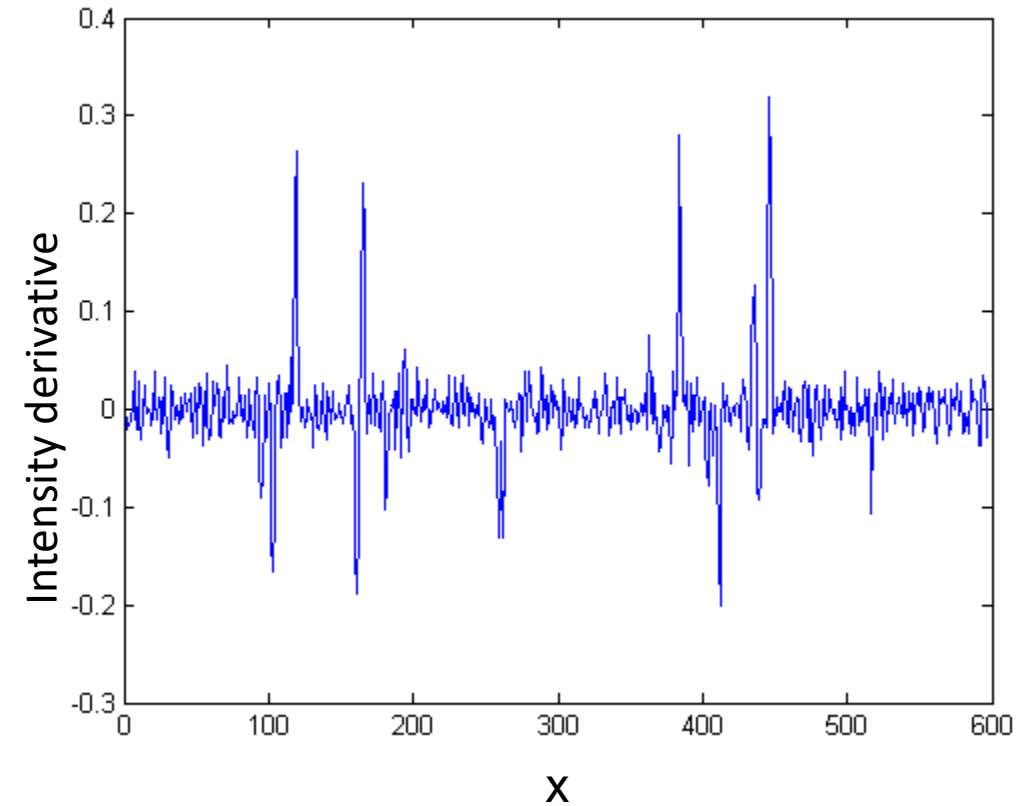
Source: D. Hoiem



Intensity Profile with a little Gaussian noise



The previous slide, showed an image with little noise. However, real-life images have varying degrees of noise in the signal. In this image we have noise, and you can see that in the derivative image it becomes harder to differentiate legitimate edges from false edges.

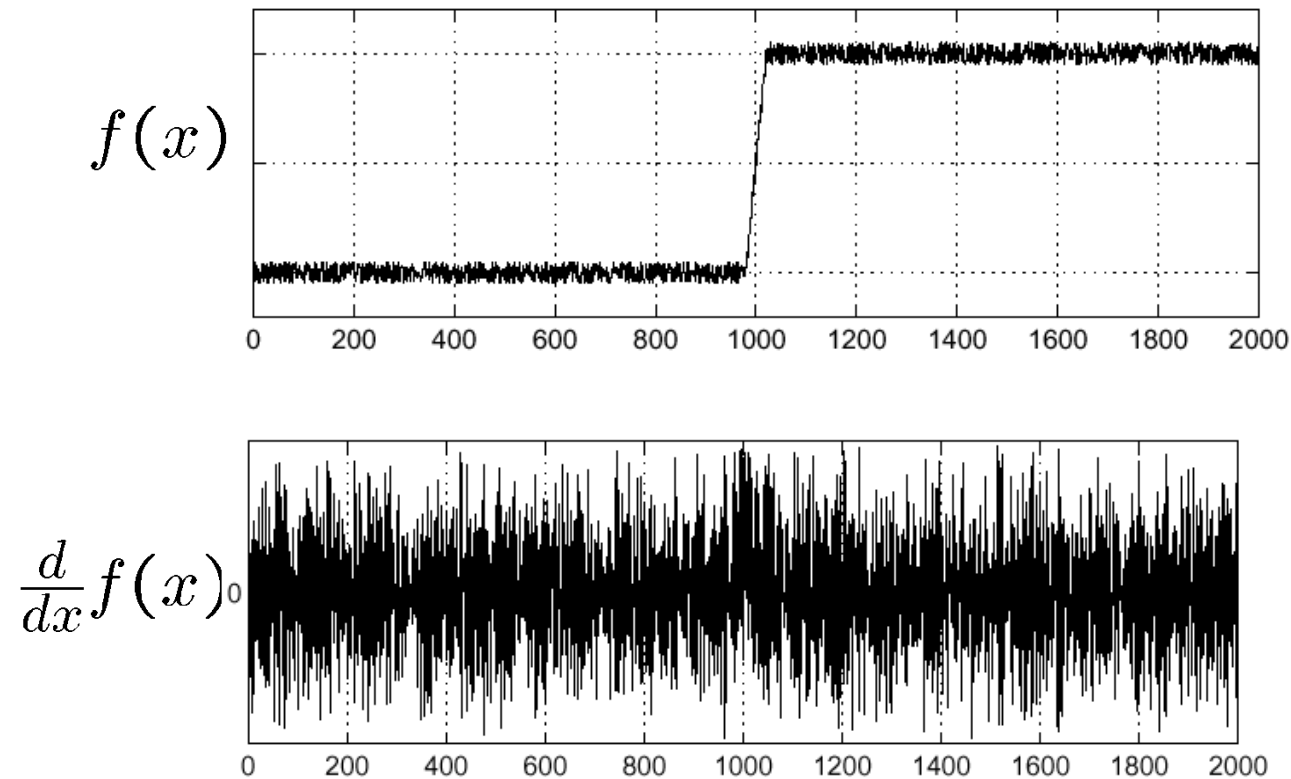


Source: D. Hoiem

Effects of noise on edge detection

Consider a single row or column of a noisy image

- Plotting intensity as a function of position gives a signal



Where is the edge?

Source: S. Seitz

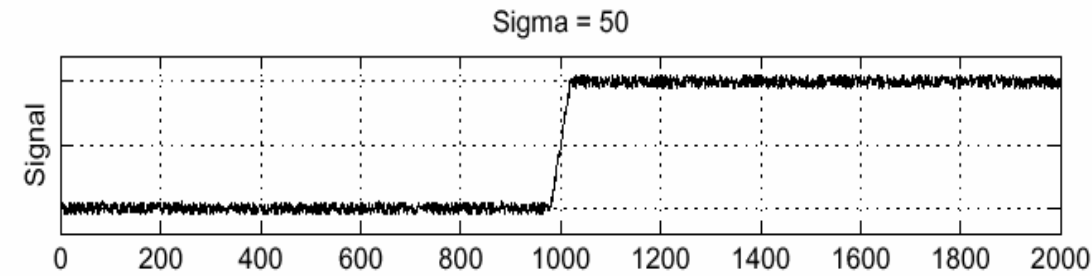
Effects of noise on edge detection

- Difference filters (like the derivative filter) are strongly sensitive to noise
 - Image noise results in pixels that look very different from their neighbors
 - Generally, the larger the noise the stronger the response
- What can we do about it?

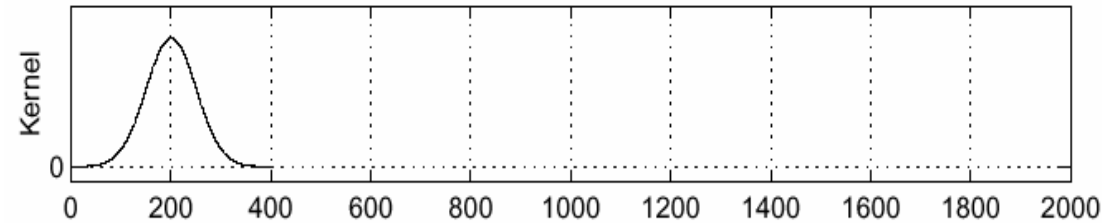
Source: D. Forsyth

Solution: smooth first

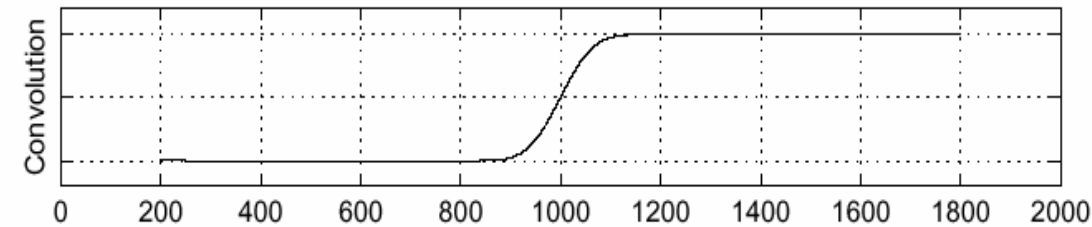
Signal: f



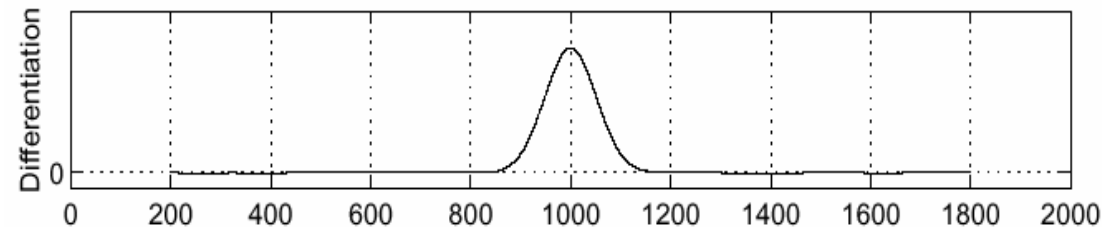
Gaussian filter: g



Apply filter: $f * g$



Compute derivative: $\frac{d}{dx}(f * g)$



- To find edges, look for peaks in $\frac{d}{dx}(f * g)$

Source: S. Seitz

Common 2D derivative filter: Sobel Filter

Choose a derivative filter

$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

What is this filter called?

Run filter over image

$$\frac{\partial f}{\partial x} = S_x \otimes f$$

$$\frac{\partial f}{\partial y} = S_y \otimes f$$

What are the dimensions?

Image gradient

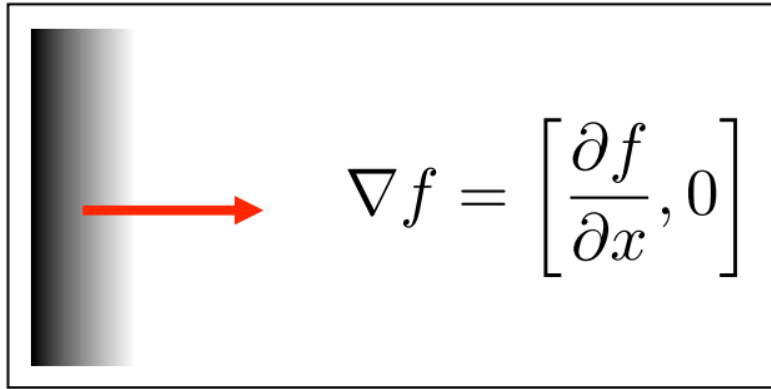
$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

What are the dimensions?

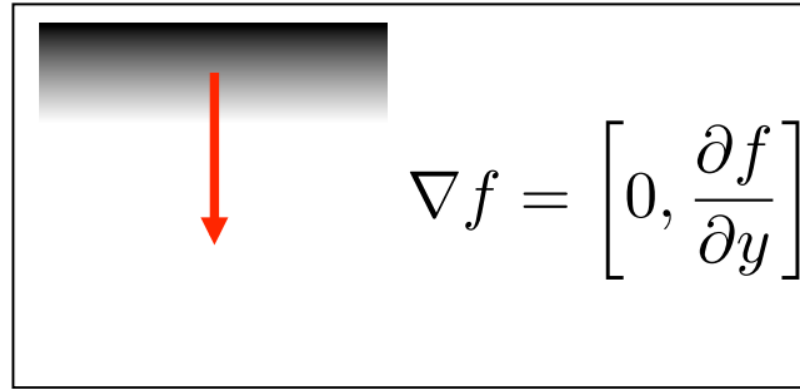
Adapted from: [link](#)

Image Gradient

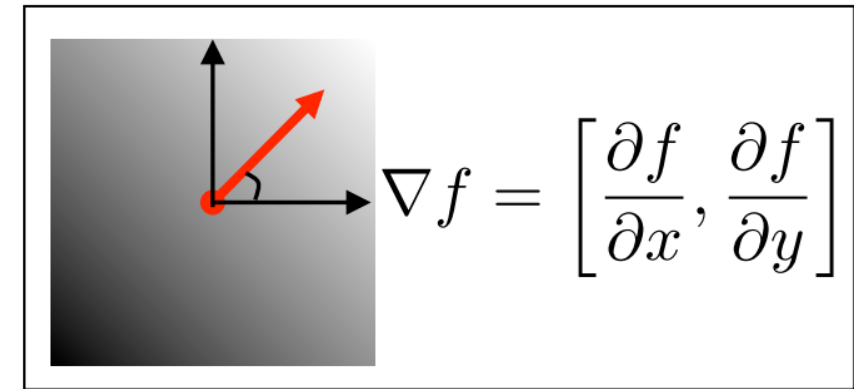
Gradient in x only



Gradient in y only



Gradient in both x and y



Gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

What does a large magnitude look like in the image?

Adapted from: [link](#)

Image Gradient

How to visualize edges using derivative filter:

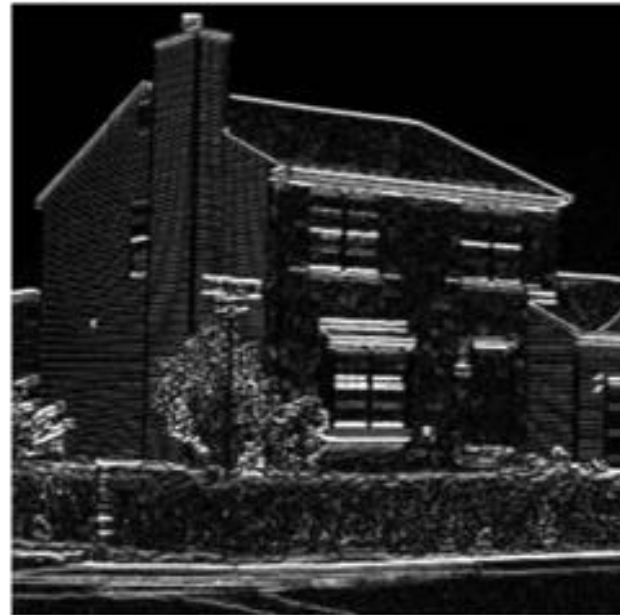
1. Consider applying smoothing as a pre-processing step to remove noise.
2. Apply derivative filter.
3. Compute derivative magnitude.
4. Make a new image that is set as the derivative magnitude.
5. Normalize the new image in the range [0,1].
6. Plot the new image to see the edges.



Original



G_x



G_y



$|G(x,y)|$

Image from: [link](#)

Applying Sobel Filter using scikit-image

```
import numpy as np
import scipy.signal as sig
from skimage import io, filters
import matplotlib.pyplot as plt

image = io.imread('./kfupm.jpg', as_gray=True) # read image as grayscale

edges = filters.sobel(image) # apply sobel filter and compute edge magnitude

# plot
fig, axes = plt.subplots(1, 2, figsize=(9, 6), constrained_layout=True)
ax = axes.ravel()
ax[0].imshow(image, cmap=plt.cm.gray)
ax[1].imshow(edges, cmap=plt.cm.gray)
plt.show()
```



Other Common 2D derivative filters

Sobel

1	0	-1
2	0	-2
1	0	-1

1	2	1
0	0	0
-1	-2	-1

Scharr

3	0	-3
10	0	-10
3	0	-3

3	10	3
0	0	0
-3	-10	-3

Prewitt

1	0	-1
1	0	-1
1	0	-1

1	1	1
0	0	0
-1	-1	-1

Roberts

0	1
-1	0

1	0
0	-1

Adapted from: [link](#)

Use of edge detection in iris segmentation

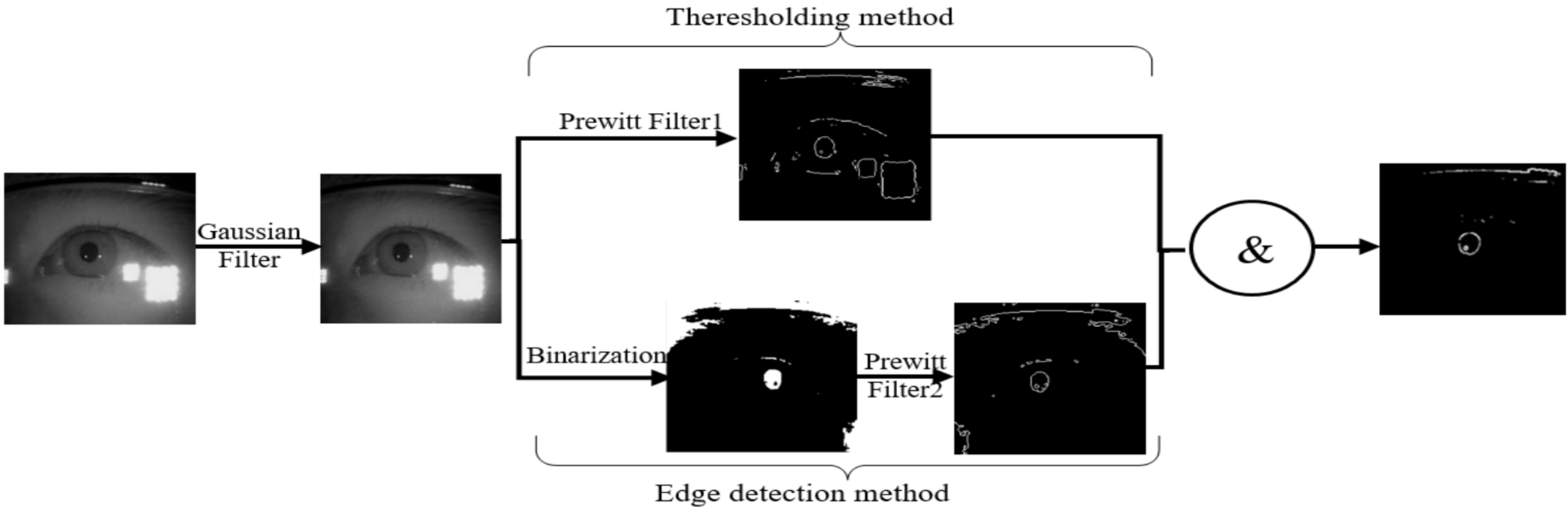


Image from: [link](#)

Segmentation of Drilled Holes

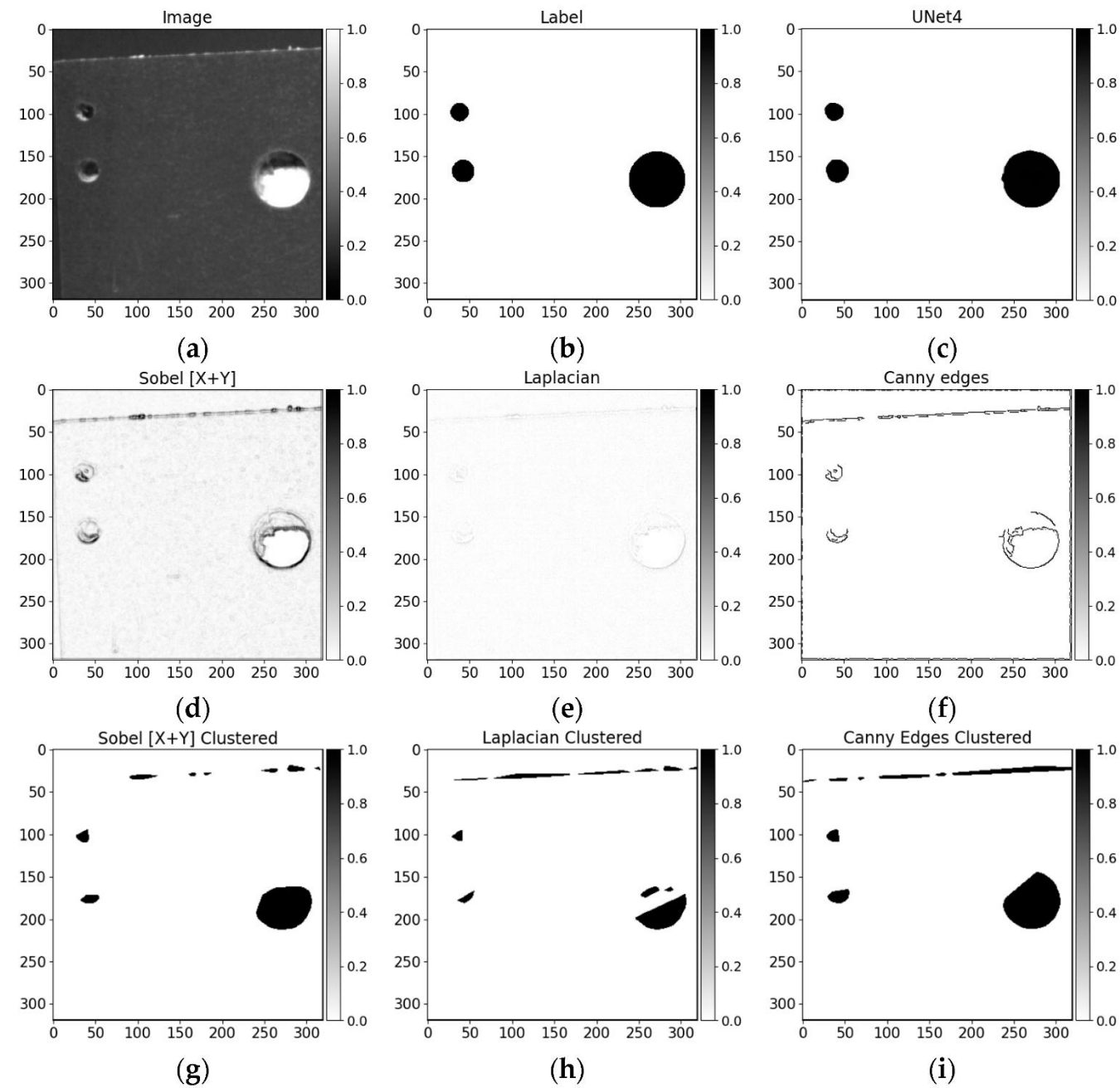
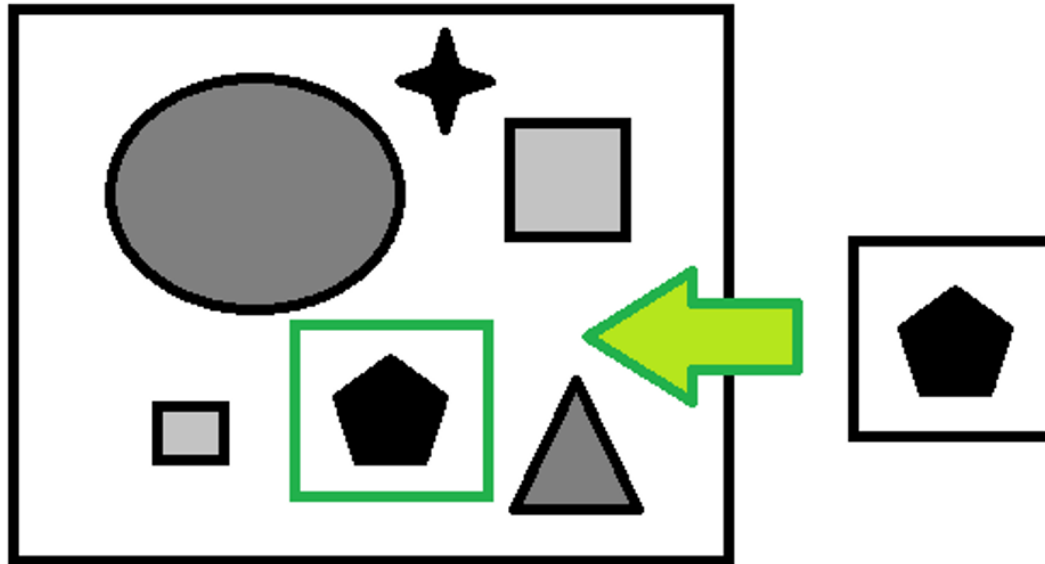


Image from: [link](#)

Template Matching

Template Matching Intro

- Template matching is a technique for finding parts of an image that match a particular template/pattern.
- Used in detecting certain objects or patterns in an image that can then be further processed.
- A simple method for template matching is to use cross-correlation with the template as the filter.



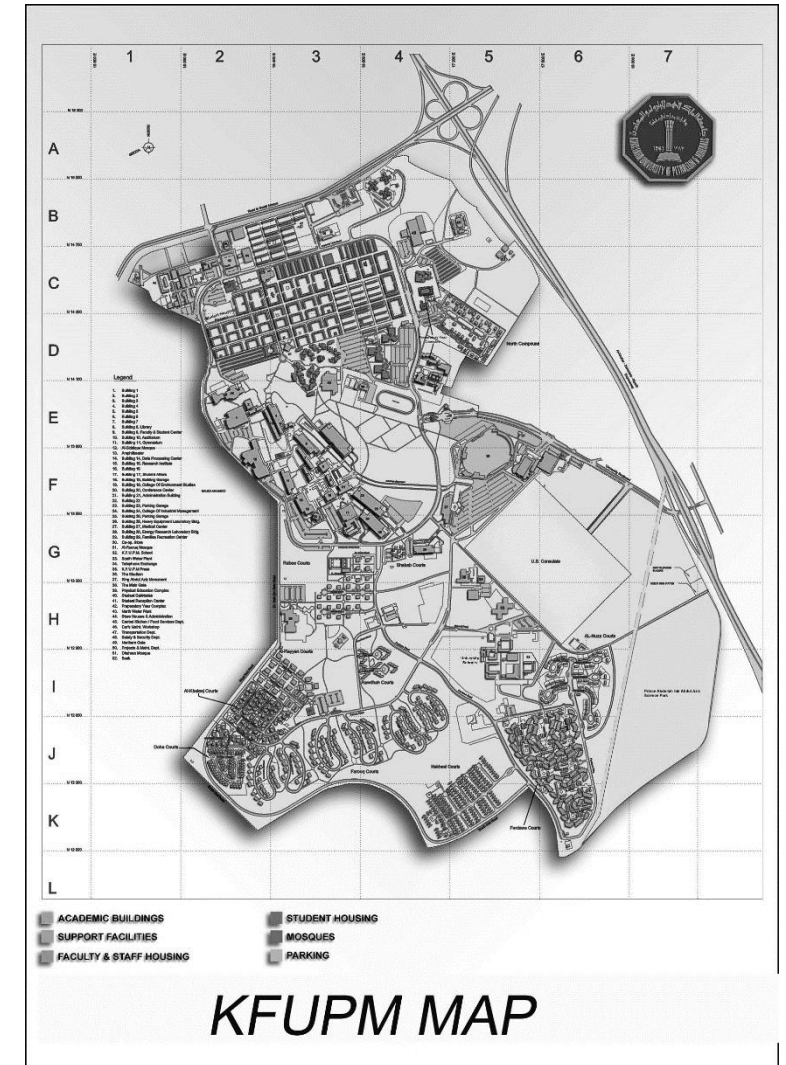
Here the template is the pentagon,
the goal is to find the pentagon in the big image.

Template Matching Using Cross-correlation [1]

- Consider the following task:
 - We have an image of a KFUPM map.
 - The template is KFUPM logo.
 - We want to detect the KFUPM logo in the image.
- Idea: apply cross-correlation with the logo as the filter so that when the filter passes over the logo in the map, the resulting number is maximized.
- How to do this?
- Problem: If we apply cross-correlation directly, then we would not be able to determine the location of the logo in the map. “Lighter” parts of the image map would have larger values from the cross-correlation.
- To fix this, do the following:
 - Subtract the mean from the image and template
 - Then apply cross-correlation.



Template



Image

```
import numpy as np
import scipy.signal as ssig
from skimage import io, filters
import matplotlib.pyplot as plt
from skimage.transform import rescale, resize, downscale_local_mean

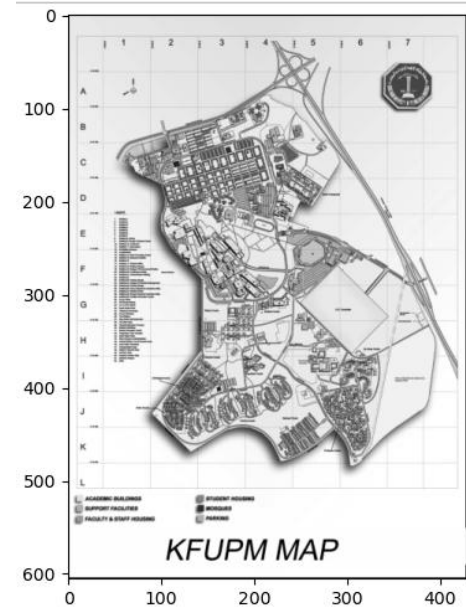
image = io.imread('./kfupm_map.jpg', as_gray=True) # read image as grayscale
image = rescale(image, 0.25)
template = io.imread('./kfupm_template.jpg', as_gray=True) # read image as grayscale
template = rescale(template, 0.25)

# without mean subtracted
res = ssig.correlate2d(image, template, mode='same')

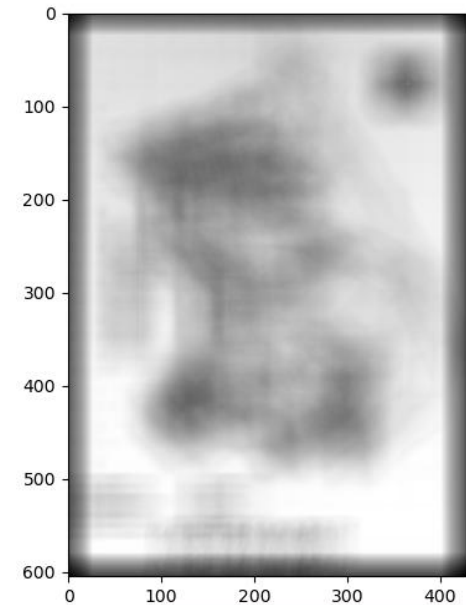
# with mean subtracted
image2 = image - np.mean(image)
template2 = template - np.mean(template)
res2 = ssig.correlate2d(image2, template2, mode='same')

# plot
fig, axes = plt.subplots(2, 2, figsize=(6, 6), constrained_layout=True)
ax = axes.ravel()
ax[0].imshow(image, cmap=plt.cm.gray)
ax[2].imshow(res, cmap=plt.cm.gray)
ax[1].imshow(image2, vmin=-0.7, vmax=0.7, cmap=plt.cm.gray)
ax[3].imshow(res2, cmap=plt.cm.gray)
plt.show()
```

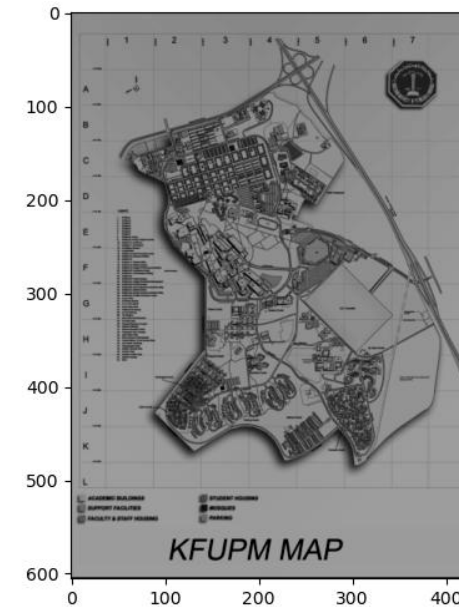
Template Matching Using Cross-correlation [3]



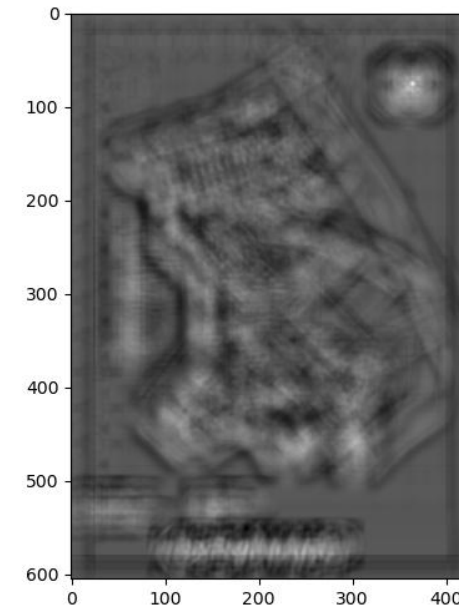
Original image (grayscale),
ranges between 0 and 1.



Result of calling
`ssig.correlate2d(image, template, mode='same')`
without subtracting the mean.



Mean subtracted image.
Ranges between -0.8 and 0.16



Result of calling
`ssig.correlate2d(image2, template2, mode='same')`
with subtracting the mean.

Notice the logo location shines brightly,
i.e. has large value.

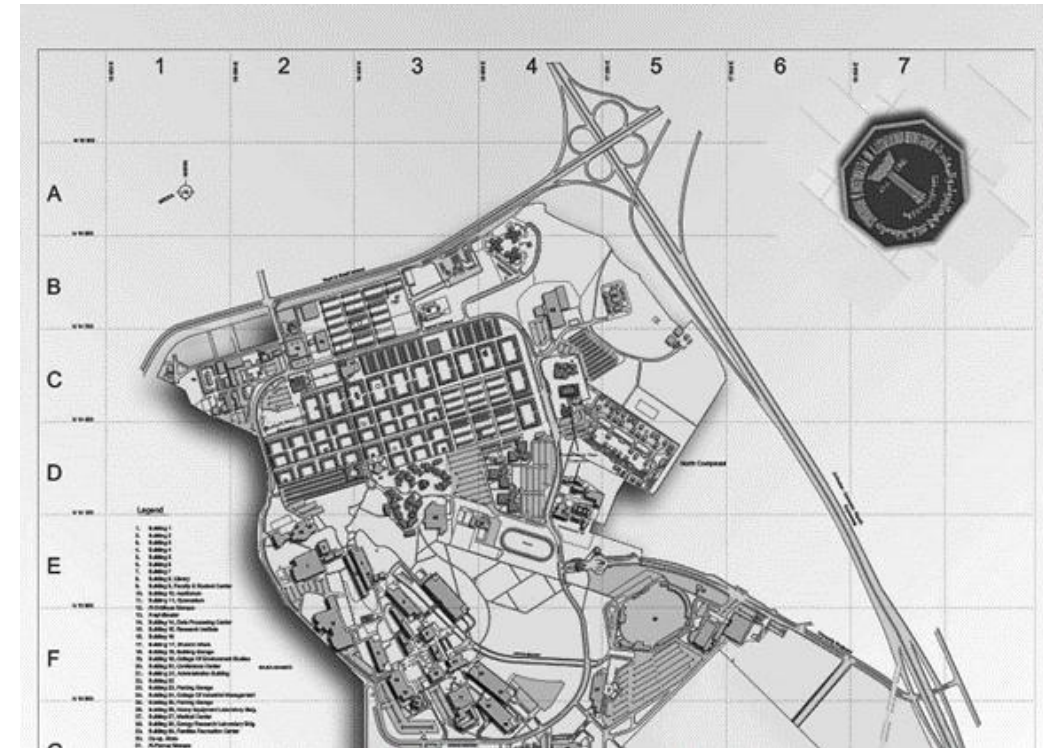
Advanced template matching – Object Recognition/Classification [Deep Learning]

Advanced Template Matching – Rotation?

- We saw that template matching can be achieved using the template as a filter in cross-correlation.
- What about when we have rotation of the template in the image?
- We can use a rotated template as a filter.
- Since we do not know the degree of rotation beforehand, we would need use the template as a filter at various degrees of rotation.



Template



Image

Advanced Template Matching – Scaled?

- What about when we have scaling (larger/smaller) of the template in the image?
- We can use a scaled template as a filter.
- Since we do not know the scaling factor beforehand, we would need use the scaled template as a filter at various scales.



Template



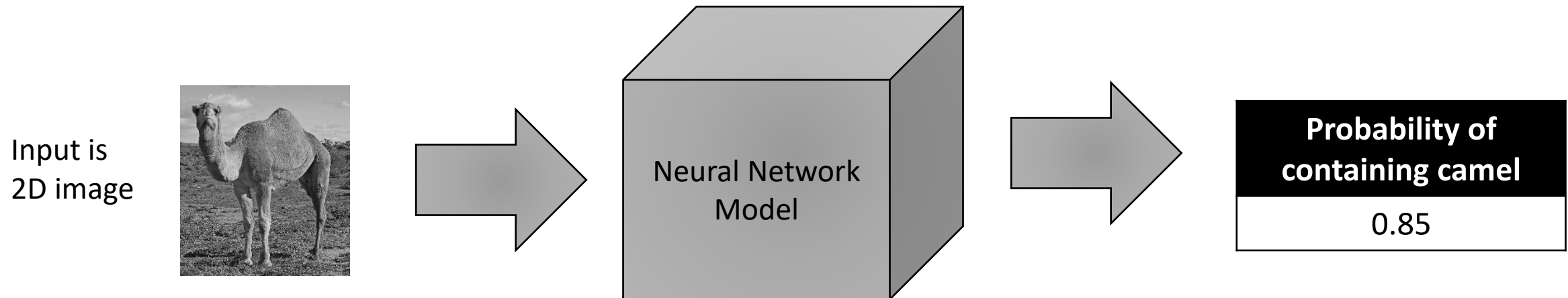
Image

Advanced Template Matching – Shearing/Rotation/Scaling/Noise?

- Is there a better way to do template matching that is not tedious?
- We want to be able to recognize the template under various conditions like rotation, scaling, noisy image, different lighting, shearing, etc.
- Idea #1: the filter is the key to template matching.
- Idea #2: can the filter **weights** be **learned** to recognize the presence of a template in an image?
- Yes, we can use machine learning with an image as input, and a yes/no label to indicate that the image has the template.
- We'll give a general overview of how this is done. We won't go into details.

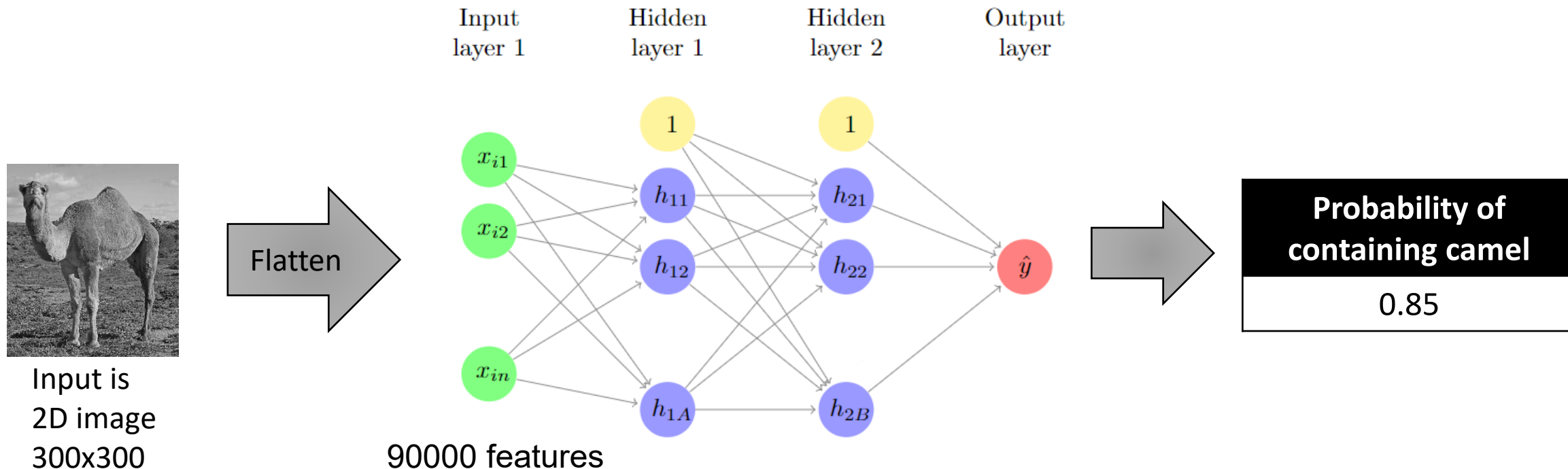
Advanced Template Matching – Learning to identify a template [1]

- Yes, we can use machine learning with an image as input, and a yes/no label to indicate that the image has the template.
- Example task: detect if an image has a camel.
 - Input: 300x300 grayscale image.
 - Label: yes/no binary label.
 - Model: neural network that outputs probability of the input image containing a camel.



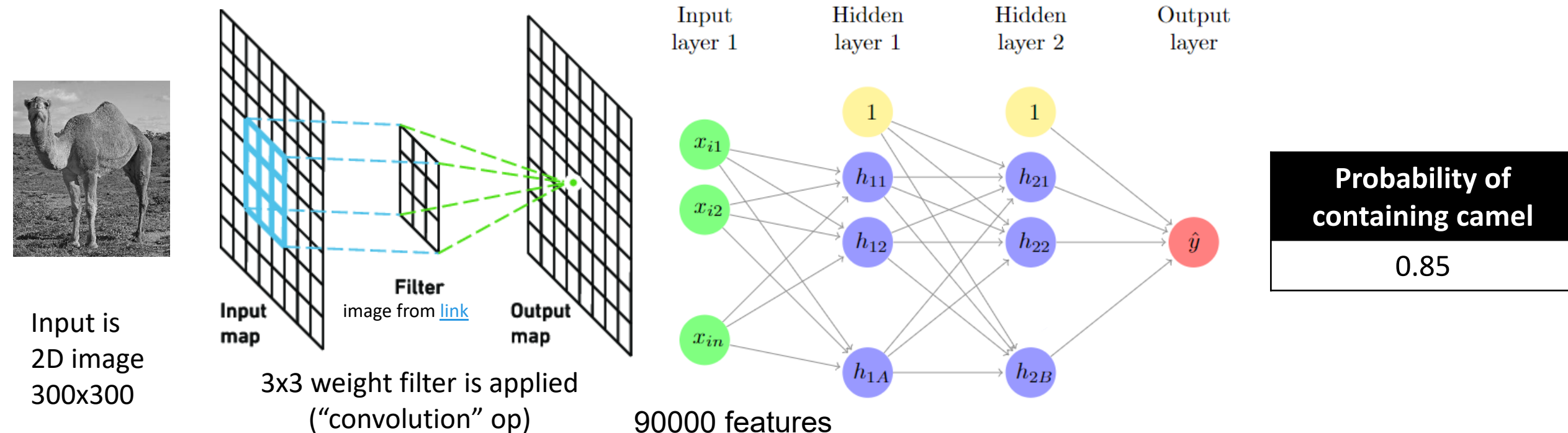
Advanced Template Matching – Learning to identify a template [2]

- Neural network architecture #1: flatten the image and use a fully-connected neural network.
- Disadvantages of this approach:
 - Large number of weight parameters to be learned. Increases model complexity, and as a consequence, requires more data to learn a good model.
 - More importantly, image pixels have **local** spatial dependence, that is one pixel is related to its **nearby** surrounding pixels. It might be better to learn local relationships in an image.



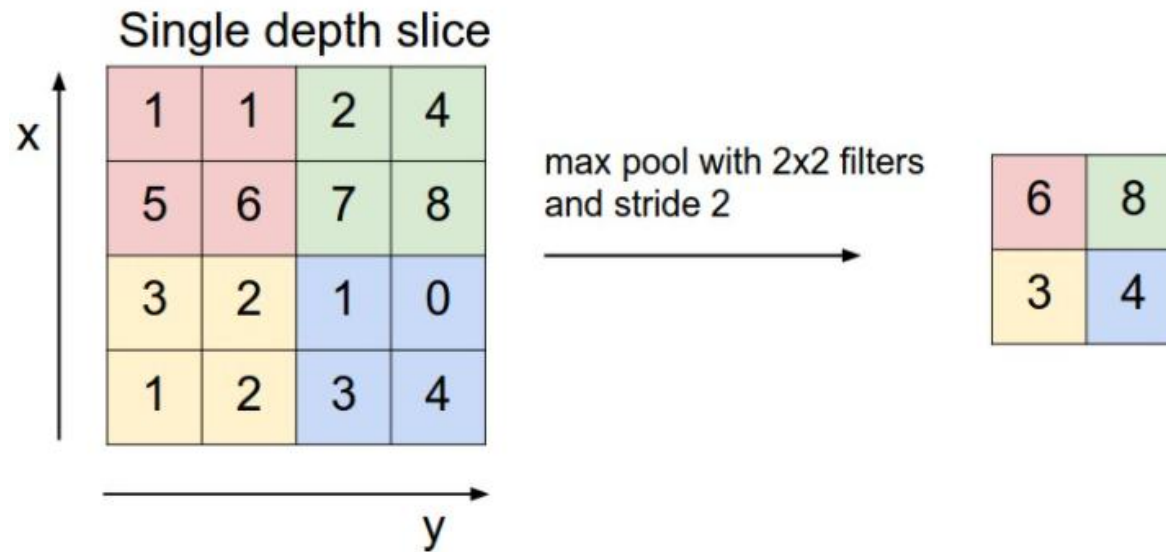
Advanced Template Matching – Learning to identify a template [3]

- Neural network architecture #2: apply cross-correlation with weight filters/kernel. Then flatten the resulting image, then apply a fully-connected network.
- Advantages of this approach: with cross-correlation, relationship among local pixels is learned.
- Disadvantage: learning a single filter may not be enough to learn the target concept. Furthermore, no reduction in image size still leads to large number of parameters in the FCN layers.

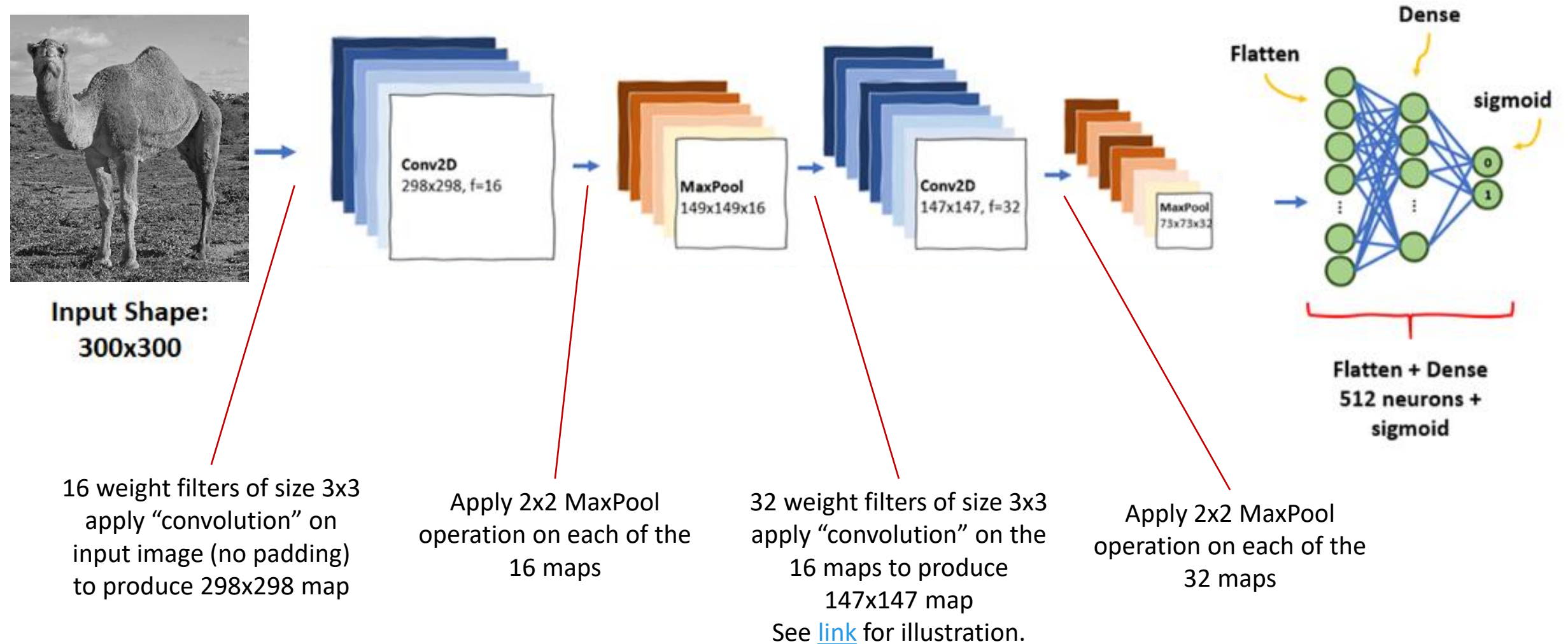


Advanced Template Matching – Learning to identify a template [4]

- Neural network architecture #3: Learn multiple weight filters, apply non-linear functions like ReLu to individual pixels, use **summarizing** operations to reduce size of maps, then finally apply flattening and fully-connected layers.
- These types of networks are called **convolutional neural networks (CNN)**.
- The cross-correlation operation is called as “convolution” in deep learning.
- Many types of summarizing operations; one common method is called Max-Pool.



Advanced Template Matching – Learning to identify a template [5]



Advanced Template Matching – Learning to identify a template [6]

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
```

```
model = tf.keras.models.Sequential([
```

```
# This is the first convolution, input_shape= 300x300 image
tf.keras.layers.Conv2D(16, (3,3), activation='relu', input_shape=(300, 300, 1)),
tf.keras.layers.MaxPooling2D(2, 2),
```

```
# The second convolution
tf.keras.layers.Conv2D(32, (3,3), activation='relu'),
tf.keras.layers.MaxPooling2D(2,2),
```

```
# flatten layer then connect FCN layers
tf.keras.layers.Flatten(),
tf.keras.layers.Dense(512, activation='relu'), # 512 neuron hidden layer
# binary classification output
tf.keras.layers.Dense(1, activation='sigmoid')
```

```
])
```

```
# to get the summary of the model
print(model.summary())
```

```
# configure the model for training by adding metrics
model.compile(loss='binary_crossentropy', optimizer=Adam(lr=0.001), metrics=['accuracy'])
```



Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 298, 298, 16)	160
max_pooling2d (MaxPooling2D)	(None, 149, 149, 16)	0
conv2d_1 (Conv2D)	(None, 147, 147, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 73, 73, 32)	0
flatten (Flatten)	(None, 170528)	0
dense (Dense)	(None, 512)	87310848
dense_1 (Dense)	(None, 1)	513
Total params: 87,316,161		
Trainable params: 87,316,161		
Non-trainable params: 0		
None		

What is being learned in these intermediate layer maps? [1]

- The intermediate maps are also 2D images that can be plotted. However, it can be difficult to interpret them.
- See below a CNN trained to predict a multi-class task (car, truck, airplane, ship, horse).
- Notice the final POOL layer is sparse; i.e. few highly active pixels.

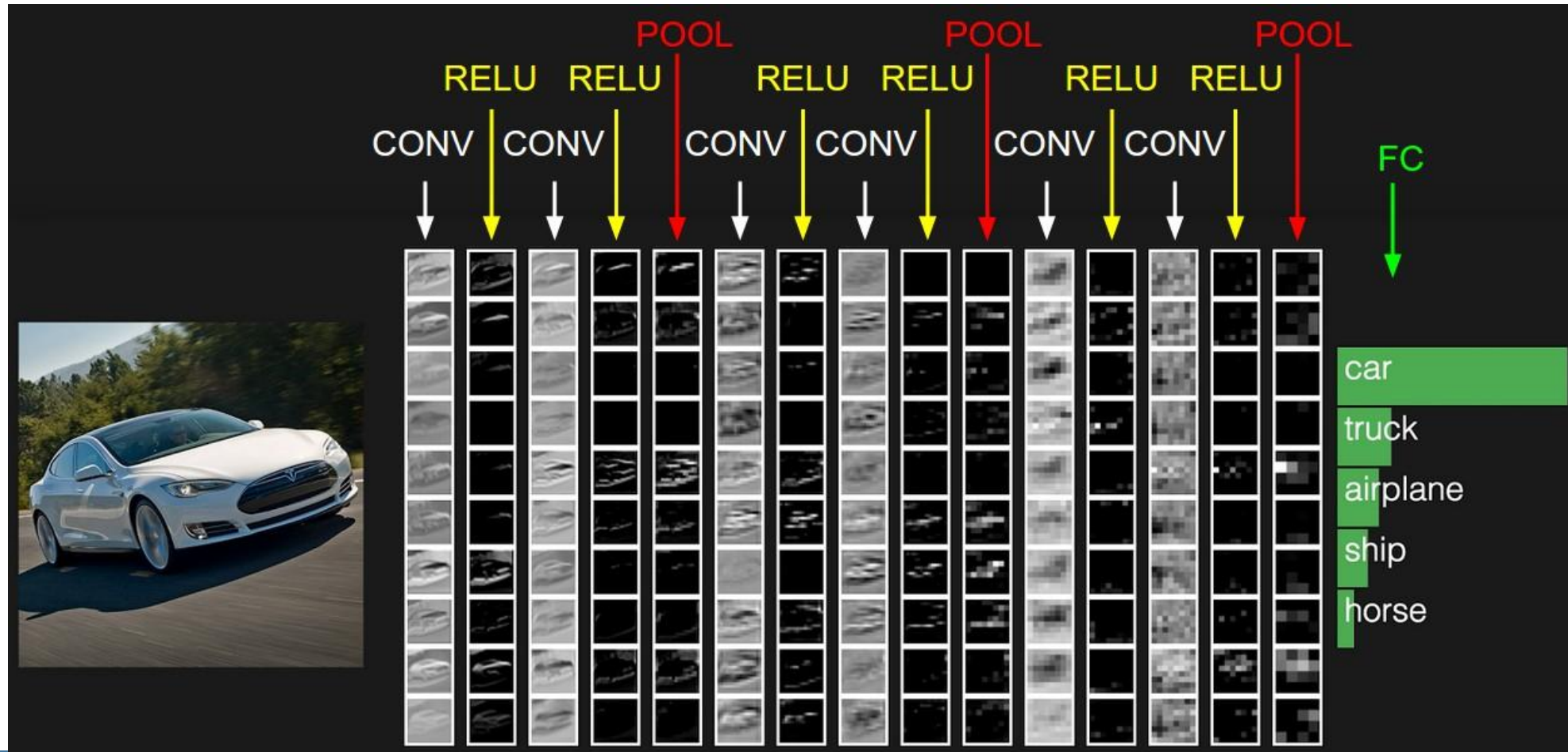


Image from: [link](#)

Advanced applications: Image/Object Segmentation

- An area of research in deep learning is interested in using CNNs to **segment** an input image.
- The CNN is trained with input images and output images with segmented classes.

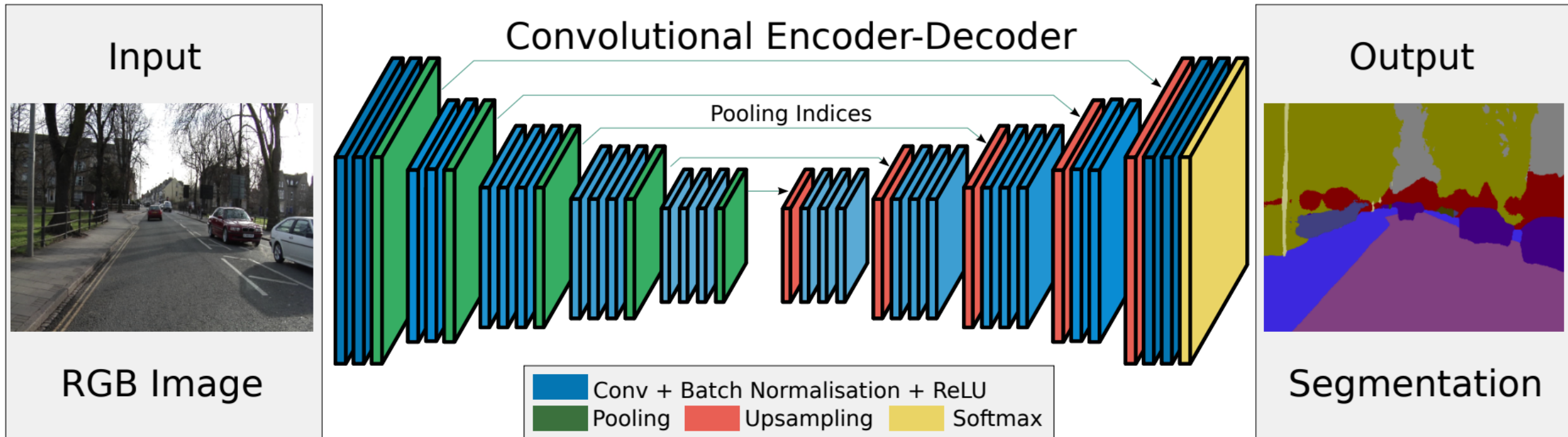


Image from: [link](#)

Advanced applications: Image/Object Segmentation



	Bicyclist
	Pedestrian
	Car
	Fence
	SignSymbol
	Tree
	Pavement
	Road
	Pole
	Building
	Sky

Image from:
[link](#)

Advanced applications: Image/Object Segmentation [Video](#)



Summary

Summary [1]

- Computer vision is a broad field of AI that encapsulates problems related to acquiring and processing the world as it is perceived.
- Image processing is concerned with tasks of analyzing acquired images and extracting information.
- Cross-correlation and convolution operations are fundamental techniques for a wide-range of image processing tasks:
 - Edge detection.
 - Template matching.
 - Advanced deep learning techniques.
- Convolution is cross-correlation with a flipped filter.
- In deep learning, the cross-correlation operation is commonly referred to as “convolution”.

Summary [2]

- Although template matching using the template as a filter is a simple technique, it cannot generalize to various transformations of the template in the input image. Rotation, scaling, shearing, noise, etc are all possible transformations that can be found in the input image.
- Convolutional neural networks (CNN) are trained to learn the filter weights using data. Thus, it can learn to distinguish the template under various circumstances.
- Using CNNs, the initial layers act as feature extractors for the target concept. In a sense, the network learns an algorithm for detecting a pattern.
- Using image data, a wide range of applications can be solved using deep learning like image segmentation.